Revisiting ContextAgent: A Reproducibility Study of Context-Aware Proactive LLM Agents under Resource Constraints

Anonymous authors
Paper under double-blind review

Abstract

Proactive Large Language Model (LLM) agents decide on their own when to offer assistance, which requires them to evaluate context continuously and makes their inference cost a first-order concern. We reproduce *ContextAgent* (Yang et al., 2025b), a context-aware proactive agent evaluated on *ContextAgentBench*, with the authors' released code and data, and test whether its results hold when the reasoner is made cheaper. The central claim, that supervised fine-tuning with persona context and thought traces outperforms in-context learning, is confirmed for all three base models, and the in-context baselines reproduce closely. The absolute fine-tuned scores do not reproduce: the paper and the released configuration state different learning rates (10^{-4} versus 5×10^{-5}), and with the paper's value tool-calling F1 rises from the reported 0.626/0.647/0.645 to 0.777/0.685/0.740 (three seeds) for Llama-3.1-8B, DeepSeek-R1-7B and Qwen2.5-7B, while validation-based selection prefers an even higher rate. Under efficiency constraints, the learning rate matters more than the adapter configuration; 4-bit quantisation halves inference memory without a detectable loss but adds up to 53% latency; and a 1.5B model reaches the released 7B score at a quarter of the memory, although 1–1.5B models stay 0.03–0.10 F1 below 7–8B models trained the same way. Pruning exposes a pitfall of the usual recovery step: merging an unconstrained adapter and re-applying the mask discards half of the update at 50% sparsity, and restricting the update to the unpruned weights removes most of the loss, but without sparse kernels pruning saves neither memory nor latency. Finally, compressed proactive agents can fail silently: outputs without a parseable proactive score are scored as decisions not to act, which keeps aggregate scores deceptively high. We release code, configurations, predictions and score files, and derive recommendations for evaluating efficient proactive agents.

1 Introduction

Most LLM-based agents are reactive: they produce an answer or perform an action only after receiving a prompt from the user. A different line of work studies proactive agents, which use available context to decide whether assistance is needed and may act without being explicitly instructed to do so (Lu et al., 2025). This changes the role of the agent from answering requests to taking the initiative, which also affects how users perceive and work with the system (Diebel et al., 2025). It also changes the practical cost of running the system. A reactive agent performs inference only on user interaction, whereas a proactive agent runs continuously or at regular intervals to monitor context, decide whether intervention is needed and possibly call external tools. A proactive agent may therefore perform well on a benchmark while still being too costly to operate in practice.

ContextAgent (Yang et al., 2025b) is a recent framework for context-aware proactive LLM agents that combines sensory context, persona information, proactive prediction and tool calling. It is accompanied by *ContextAgentBench*, a benchmark with 1,000 samples across nine daily scenarios and 20 tools, and by a public implementation. The original work focuses on capability: it shows that supervised fine-tuning of 7–

8B models with persona context and thought traces outperforms in-context learning baselines on proactive prediction and tool calling. The cost of maintaining this behaviour is not evaluated.

This paper is a reproducibility study of *ContextAgent*. We first reproduce the main results on *ContextAgentBench* with the authors' code and data split. We then test whether the conclusions hold, and at what cost, when the fine-tuned reasoner is made cheaper to train and run with widely used efficiency techniques: Low-Rank Adaptation (LoRA) configuration choices, base-model quantisation (LLM.int8 and 4-bit NormalFloat (NF4)), pruning (unstructured, semi-structured and structured) and smaller base models.

Compression is usually judged on reactive benchmarks, where a broken model answers wrongly and its score drops. A proactive agent can fail in a way that such benchmarks do not expose: if it never decides to act, or if its outputs cannot be parsed and are therefore scored as decisions not to act, it still answers every sample that needs no assistance correctly. On *ContextAgentBench*, a model that never acts reaches an Proactive Prediction Accuracy (Acc-P) of 0.470 and a tool-calling F1-score of 0.468, which is the share of samples whose correct answer is to stay passive. We observe exactly this *silent passivity* twice, both times because no output contains a parseable proactive score: for DeepSeek-R1-7B trained with an effective batch size of 64 at the released learning rate, and for structured magnitude pruning at every ratio (Table 13, Table 4). Aggregate scores alone therefore cannot distinguish a degraded proactive agent from a non-functional one, and we report the miss rate (Missed Detection rate (MD)) and the share of outputs without a parseable proactive score alongside them (Section 6.3).

Our contributions are:

- **A claim-by-claim reproduction.** The central qualitative claims reproduce for all three base models, and so do the in-context baselines. The absolute fine-tuned scores do not, and we trace this to documentation gaps in the paper and the release: a learning rate that differs between paper and code, an unstated effective batch size, a missing configuration for one of the three models, unreported seeds, and an evaluation metric whose implementation differs from its description (Table 8).
- **The effect of the learning-rate discrepancy.** With the paper's learning rate, all three models exceed the reported fine-tuned scores (Llama-3.1-8B: 0.777 versus 0.626 tool-calling F1, three seeds); with the released one, two of the three do. Validation-based selection prefers an even higher learning rate, and training with the effective batch size of eight-GPU data-parallel training reproduces the reported score of only one model (Section 6.1).
- **A statistically controlled efficiency study.** With three seeds for all central configurations, paired bootstrap intervals and a correction for multiple comparisons, we show that the learning rate matters more than the LoRA configuration, that 4-bit quantisation halves memory without a detectable loss but costs latency, that 1–1.5B models trade 0.03–0.10 F1 for a fourfold memory reduction, and that the usual merge-and-re-mask recovery step, not the sparsity itself, causes most of the loss of pruning at 50% sparsity (Section 6.2).
- **How proactive agents fail under compression.** An error cascade over the decision pipeline shows that compressed models fail first on the output format and that some configurations fall silent while keeping deceptively high aggregate scores; we derive reporting recommendations for proactive-agent benchmarks (Section 6.3, Section 7.3).

The study is limited to *ContextAgent* as the framework and *ContextAgentBench* as the evaluation setting, and it covers the context-aware reasoner only; the perception stage (vision-language model and speech recognition) is outside the cost measurements.

2 Related Work

2.1 Proactive LLM Agents

Most LLM agents are reactive systems, where the model acts after receiving an explicit user prompt. ReAct (Yao et al., 2023) combines reasoning and action in an iterative process, and Toolformer (Schick et al., 2023)

teaches models when to call external tools; both are relevant for tool use and multi-step decision making, but the agent is still given a task or observation to respond to. A proactive agent must additionally decide whether assistance should be provided at all.

Lu et al. (2025) frame proactivity as a prediction problem in which the model learns when proactive suggestions are likely to be accepted by users. Yang et al. (2025a) study proactivity in a deployed setting, where sensory context is gathered through on-demand perception and used together with LLM reasoning to provide proactive assistance on AR glasses. *ContextAgent* (Yang et al., 2025b) combines sensory context, persona information, proactive prediction and tool calling, and evaluates both whether the agent should act and how it should act. These systems primarily target proactive capability; their cost of operation is not the main subject of evaluation.

2.2 Efficient Model Adaptation and Compression

Parameter-efficient fine-tuning methods such as LoRA reduce training cost by freezing the base model and training a small set of adapter parameters (Hu et al., 2022). LoRA is more sensitive to its hyperparameters than full fine-tuning, above all to the learning rate (Biderman et al., 2024), which makes under-documented learning rates a likely source of irreproducible fine-tuning results. Quantised Low-Rank Adaptation (QLoRA) extends this approach by quantising the frozen model during training (Dettmers et al., 2023), and LLM.int8 provides 8-bit inference with outlier handling (Dettmers et al., 2022). Pruning removes parameters or structures from an existing model (Cheng et al., 2024; Sun et al., 2024; Fang et al., 2023); pruned LLMs are usually recovered with LoRA fine-tuning (Ma et al., 2023), and sparsity-preserving variants restrict the update to the unpruned weights so that merging keeps the sparsity (Lu et al., 2024; Muñoz et al., 2024), while Knowledge Distillation (KD) trains a smaller student model to imitate a larger teacher (Hinton et al., 2015); for generative models, sequence-level distillation trains the student on the teacher's outputs (Kim & Rush, 2016), and recent variants for LLMs distil on the student's own generations (Gu et al., 2024; Agarwal et al., 2024). Post-training methods quantise or prune a trained model in one shot from a small calibration set (Frantar et al., 2023; Frantar & Alistarh, 2023), and serving systems such as vLLM (Kwon et al., 2023) provide the kernels that turn reduced precision into lower latency. Surveys report these as the most common approaches for reducing the cost of LLMs (Kim et al., 2025; Dantas et al., 2025), but they are usually evaluated on reactive tasks. Whether they preserve the behaviour a proactive agent needs, namely calibrated proactive scores, a fixed structured output format and valid tool arguments, is the question this paper addresses for *ContextAgent*.

2.3 Reproducibility Studies

Independent reproduction of published machine learning results, as promoted by the ML Reproducibility Challenge (Pineau et al., 2021) and the reproducibility tracks of information retrieval venues, regularly reveals undocumented choices and discrepancies between papers and released code (Gundersen & Kjensmo, 2018). Reported results are also sensitive to random seeds and evaluation noise, so comparisons should be based on several runs and report their uncertainty (Dodge et al., 2019; Bouthillier et al., 2021); we follow this advice with three seeds for all central configurations and paired confidence intervals. Following the ACM terminology, our study is a *reproduction* (different team, same artefacts) of the *ContextAgentBench* results, extended with a *generalisation* to resource-constrained settings.

3 Background

This section introduces the proactive agent setting, the *ContextAgent* framework and the *ContextAgentBench* benchmark.

3.1 Proactive LLM Agents

A reactive agent receives an explicit instruction and produces an output. A proactive agent instead uses available context to decide whether assistance is needed and may act without being directly prompted (Lu

et al., 2025). Formally, at each step t the agent maps a contextual state c_t (observations, user information and system state) to $f(c_t) = (p_t, a_t)$, where p_t is the estimated need for assistance and a_t the planned action, a response or a sequence of tool calls. The agent stays passive if $p_t < \theta$ for a threshold θ and acts otherwise. Because this decision is evaluated repeatedly, also when the user requests nothing, the total cost of a proactive agent depends on the cost of a single decision and on how often it is made.

3.2 ContextAgent

ContextAgent (Yang et al., 2025b) instantiates this setting with sensory perception and persona context. Its first stage turns raw sensory input S (egocentric video, audio and smartphone notifications) into a textual context $C = (C_V, C_A, N)$, using a vision-language model for the visual context C_V, speech recognition for the audio context C_A, and the notifications N directly. Its second stage, the context-aware reasoner, is an LLM

$$(T, PS, TC) = A_S(C, P) \tag{1}$$

that takes the context C and persona information P and produces a thought trace T, a proactive score PS and a planned tool chain $TC = ((t_i, a_i))_{i=1}^{m}$ of tools t_i from a fixed set $\mathcal{T}$ with arguments a_i. If $PS \geq \theta$, the tools are executed in sequence and their outputs, together with the context, persona and thought trace, are used to generate the final response R. The reasoner is trained by supervised fine-tuning on samples (X_i, T_i, Y_i), where $X_i = (C_i, P_i)$ and $Y_i = (PS_i, TC_i)$. This reasoner is the object of our study: it runs at every decision step and dominates the cost of the agent.

3.3 ContextAgentBench

ContextAgentBench contains $N = 1000$ samples $(S_i, C_i, P_i, T_i, PS_i, TC_i, R_i)$ across nine daily scenarios and 20 tool types, built from 200 human-written seed examples that were expanded with LLMs and checked by annotators and a format script. The proactive score is defined on a five-point scale, $PS_i \in \{1, \ldots, 5\}$, from no need to a high need for proactivity, and a sample may require up to five tools. The original paper also introduces *ContextAgentBench-Lite*, with 300 human-verified samples paired with raw sensor data such as egocentric video and audio. The released Lite file contains these 300 samples with the text produced by the perception pipeline from the raw data; all of them are also *ContextAgentBench* samples (181 in the training and 119 in the test split) with the same labels. We evaluate on it in Section 6.1.1. For non-proactive cases, where $PS_i \in \{1, 2\}$, both the planned tool chain and final response are set to $TC_i = \texttt{None}$ and $R_i = \texttt{None}$[1].

The released benchmark comes with a fixed split of 600 training and 400 test samples, which we use throughout.

4 Methods

This section describes the reproduction protocol and the efficiency techniques applied to the *ContextAgent* reasoner.

4.1 Reproduction Protocol

We reproduce the main *ContextAgentBench* experiment of Yang et al. (2025b) (their Table 1) with the three base models reported there: Llama-3.1-8B-Instruct (Grattafiori et al., 2024), Qwen2.5-7B-Instruct (Qwen Team, 2024) and DeepSeek-R1-Distill-Qwen-7B (Guo et al., 2025), which we call DeepSeek-R1-7B as the original paper does. We use the released code, prompts, data split and evaluation scripts; deviations are listed in Section 5.3. We test the following claims:

C1 Supervised fine-tuning with thought traces and personas (*ContextAgent*) outperforms all in-context learning baselines on both proactive prediction and tool calling.

[1] In the released data, five non-proactive samples (one in the test split) nevertheless carry a tool chain, and one test sample is labelled as non-proactive with a proactive score of 5. A model that never acts therefore reaches an Acc-P of 0.470 (188 of the 400 test samples have a score below 3) but an F1-score of 0.468 (187 have no ground-truth tool).

C2 Adding persona context and thought traces to in-context learning (ICL-All) improves over Vanilla ICL.

C3 The reported absolute scores can be obtained with the released artefacts.

The further claims of the original paper (*ContextAgentBench-Lite*, out-of-domain scenarios, missing sensors and proprietary models) are summarised with their reproduction status in Table 8. All models are evaluated with the original metrics and evaluation code: Acc-P, MD, False Detection rate (FD) and Root Mean Square Error (RMSE) for proactive prediction, and precision, recall, F1-score and Argument Accuracy (Acc-Args) for tool calling (Section 5.6). For the configurations behind the claims (the released learning rate 5×10^{-5} and the paper's 10^{-4}), we train three seeds each and compare configurations with a paired bootstrap over the 400 test samples (Section 5.6); to explain deviations from the reported scores, we additionally test the effective batch size of eight-GPU data-parallel training (Section 5.3).

4.2 Efficiency Techniques

We study the techniques that surveys identify as the most common ways to reduce the cost of LLMs (Kim et al., 2025; Dantas et al., 2025): the configuration of parameter-efficient fine-tuning, base-model quantisation, pruning, and smaller base models with and without knowledge distillation. Each is well studied on reactive tasks; our question is whether it preserves what the *ContextAgent* reasoner needs, namely calibrated proactive scores, a fixed structured output format and valid tool calls.

LoRA configuration. *ContextAgent* already fine-tunes its reasoner with LoRA (Hu et al., 2022), which trains a low-rank update $\Delta W = BA$ ($B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$) next to each frozen weight matrix W_0. We therefore do not compare against full fine-tuning, but vary the rank r, the targeted modules (attention, Multilayer Perceptron (MLP) or all linear layers) and the learning rate, to which LoRA is known to be particularly sensitive (Biderman et al., 2024).

Quantisation. We quantise the frozen base model and keep 16-bit adapters (QLoRA; Dettmers et al., 2023), with 8-bit LLM.int8, which processes outlier features in 16-bit precision (Dettmers et al., 2022), and 4-bit NF4 with double quantisation (Dettmers et al., 2023). The same quantised base model is used for training and inference.

Pruning. Given parameters $\boldsymbol{\theta}$ and a binary mask $\mathbf{m}$, the pruned model is $f(x; \mathbf{m} \odot \boldsymbol{\theta})$ (Cheng et al., 2024). For unstructured pruning we compare magnitude pruning with Pruning by **W**eights **and A**ctivations (WANDA) (Sun et al., 2024), which scores each weight of a linear layer by $S_{ij} = |w_{ij}| \cdot \|\mathbf{X}_j\|_2$, the product of its magnitude and the norm of its input feature over calibration data, and removes the lowest-scoring weights per output row. Semi-structured N:M pruning applies the same scores but keeps only the N highest-scoring weights in every block of M (2:4 and 4:8), a pattern that sparse tensor cores can execute (Mishra et al., 2021). Structured pruning removes whole hidden dimensions of the MLP layers with the dependency-graph framework of Fang et al. (2023), ranking groups either by the L2 norm of their weights (magnitude) or by calibration activations; attention heads are kept because pruning them degraded the model severely in preliminary experiments.

Recovery after pruning. Pruned models are recovered by LoRA fine-tuning on the task data, as is common for pruned LLMs (Ma et al., 2023). Merging a LoRA adapter, however, adds the dense update BA to every weight and thereby fills the pruned positions. We study two ways of keeping the sparsity: re-applying the mask after merging (*re-masked* recovery), which is what a standard merge pipeline yields once the sparsity is enforced, and restricting the update to the unpruned weights during training (*mask-aware* recovery), $\Delta W = (BA) \odot M$ with $M = \mathbb{1}[W \neq 0]$. Sparsity-preserving fine-tuning of this kind has been proposed before (Lu et al., 2024; Muñoz et al., 2024); we use the simplest variant to quantify how much of the apparent cost of pruning in this task is caused by the recovery step.

Smaller base models and distillation. We fine-tune 1–1.5B models of the same three families directly and compare this with distilling the fine-tuned 7–8B models into them: sequence-level distillation, where

the student is trained on the teacher's outputs (Kim & Rush, 2016), with and without filtering of incorrect teacher outputs.

5 Experimental Setup

5.1 Data and Configurations

All experiments use the released *ContextAgentBench* split of 600 training samples (`cab_train_sft`) and 400 test samples (`cab_test`) (Yang et al., 2025b). Each sample provides the context information extracted from the sensors, a persona, a reference thought trace, the proactive score, the tool chain with arguments and the final response (Section 3.3).

The combinations of these fields define the methods in Table 1, following the original paper. *Vanilla ICL* uses the context information only; *CoT* adds thought traces; *ICL-P* adds personas; and *ICL-All* adds both. *ContextAgent* is the supervised configuration: the model is fine-tuned with LoRA adapters on all linear layers using sensory context, thought traces and personas, which makes it the fine-tuned counterpart of *ICL-All*.

5.2 Implementation Details

All fine-tuning uses the LLaMA-Factory (Zheng et al., 2024) configuration released with *ContextAgent*, with a sequence cut-off of 2048 tokens, 5 epochs, a cosine learning-rate schedule with 10% warm-up, bfloat16 mixed precision, gradient checkpointing and an effective batch size of 8 (per-device batch size 1, 8 gradient-accumulation steps). The default LoRA configuration is rank $r = 8$, $\alpha = 2r$, all linear layers, and learning rate 5×10^{-5}, as in the released code. Experiments ran on a single NVIDIA L4 GPU with 24 GB of memory. The software environment used PyTorch 2.7.0, Transformers 4.52.1, PEFT 0.15.2, LLaMA-Factory 0.9.3.dev0, and bitsandbytes 0.45.1. Quantised runs used the bitsandbytes backend for 4-bit quantisation. The training and evaluation runs reported in this paper took about 550 GPU-hours on single L4 GPUs (logged per run in our release); including development, failed and repeated runs, the study used roughly 900 GPU-hours. Generation used top-p/top-k sampling with $p = 0.7$ and $k = 50$, temperature 0.95, one beam, and a maximum of 1024 new tokens. Yang et al. (2025b) report no repeated seeds or variance. We train three seeds (42, 1 and 2) for every configuration that supports a claim: the released and the paper's learning rate, the combined configurations, mask-aware pruning recovery at 10^{-4}, the 1–1.5B models with $r = 32$ (including 4-bit Qwen2.5-1.5B), the distilled students and the score-first models; the remaining sweeps (rank, target modules, further learning rates, quantisation at the default configuration) are single runs. The seed-to-seed standard deviation of tool-calling F1 is mostly 0.005–0.015, smaller than the evaluation uncertainty of a single run. With 400 test samples, a single evaluation has a 95% bootstrap confidence interval of about ± 0.035 for the tool-calling F1-score and about ± 0.05 for Acc-Args; we test differences between configurations explicitly (Section 5.6) rather than interpreting differences of a few hundredths.

5.3 Deviations from the Original Setup

- **Learning rate discrepancy.** The ContextAgent paper reports an SFT learning rate of 10^{-4} (their Section 5.1), whereas the released LLaMA-Factory training configuration files (`llama3_lora_sft.yaml`, `qwen_lora_sft.yaml`) specify 5×10^{-5}. Rather than assuming an error, we treat this as a paper–code discrepancy and evaluate both settings: we adopt the released configuration (5×10^{-5}) for our baseline reproduction and evaluate the paper's stated value (10^{-4}) in a separate learning-rate experiment (Table 2, Table 17). With the paper's value, all three models exceed the reported scores; with the released value, Llama and Qwen exceed them while DeepSeek falls below (0.596 versus 0.647). 10^{-4} improves over 5×10^{-5} for all three models.
- **Hardware and batch size.** The paper reports training on eight A6000 GPUs. The released configuration uses per-device batch size 1 with 8 gradient-accumulation steps; if it was run data-parallel on eight GPUs, the effective batch size was 64 rather than 8, corresponding to eight times fewer optimiser steps. The released training script, however, restricts training to a single GPU (`CUDA_VISIBLE_DEVICES=0`), so an effective batch size of 64 would require a setup that differs from

the release. We train on a single GPU with an effective batch size of 8. To test whether this explains the difference to the reported scores, we additionally train each model with an effective batch size of 64 (per-device batch size 1, 64 accumulation steps), which reproduces the optimiser-step count of eight-GPU data-parallel training (Table 13).

- **Fine-tuning details.** Our LoRA and learning-rate experiments add a LoRA dropout of 0.05, which the released configuration does not set (LLaMA-Factory default 0); gradient checkpointing and the LoRA scaling $\alpha = 2r$ are LLaMA-Factory defaults, and all other hyperparameters follow the released files. The release contains training configurations only for Llama and Qwen, although DeepSeek-R1-7B is one of the three models of the main results; for DeepSeek we use the same settings with LLaMA-Factory's `deepseek3` chat template (one of the released Lite scripts refers to a DeepSeek configuration that is not in the repository). The quantised (QLoRA) runs use per-device batch size 2 with 4 accumulation steps; the effective batch size of 8 is unchanged. To check that these settings do not affect the results, we also trained each model twice with the exact released configuration (for DeepSeek, the released Llama/Qwen settings with the `deepseek` template). For DeepSeek and Qwen, these runs are equivalent to ours within ± 0.03 in F1-score (differences -0.004 and 0.000; for DeepSeek on all metrics). For Llama, the released configuration scores slightly lower (-0.021, 95% CI $[-0.042, 0.000]$), a difference at the edge of significance and far smaller than the effects discussed below (Table 26).
- **Random seeds.** The released configuration sets no seed, so LLaMA-Factory's default (42) applies. Our original runs used this default. For every configuration that supports a claim (Section 5.2) we additionally train seeds 1 and 2 and report the mean and standard deviation over the three runs.
- **Configuration search.** The released split has no validation set. The LoRA sweeps in Section 6.2 are therefore evaluated on the test split and should be read as an oracle sensitivity analysis; the "best" configurations are optimistic estimates and are not used to support the reproduction claims, which rely only on the released and paper configurations. To check this selection, we repeat the configuration grid on a validation split of the training data (Section 6.2.1).

5.4 Quantisation Settings

Quantised runs load the base model with bitsandbytes before injecting 16-bit adapters. The 8-bit runs use LLM.int8 (Dettmers et al., 2022); the 4-bit runs use NF4 with double quantisation (Dettmers et al., 2023). The trainable adapters are kept in bfloat16. All other parameters are identical to the corresponding 16-bit run.

5.5 Pruning Settings

Pruning is evaluated on `Llama-3.1-8B-Instruct`, the strongest model in our reproduction, at sparsity ratios of 10%, 30% and 50% for unstructured and structured pruning, and with 2:4 and 4:8 patterns for semi-structured pruning (Appendix D). WANDA and magnitude pruning are compared for unstructured and semi-structured sparsity, while structured pruning uses group magnitude and activation-based importance. Data-aware criteria use calibration samples from the training split with a fixed sequence length of 2048 tokens: 256 samples for WANDA and 512 for activation-based structured pruning (the defaults of the respective implementations).

Evaluations immediately after pruning showed severe degradation at moderate and high sparsity, so all pruned models undergo the same LoRA-based recovery fine-tuning before evaluation with the default configuration ($r = 8$, 5×10^{-5}). The structured models were originally recovered with a learning rate of 2×10^{-5}; we repeat them with 5×10^{-5} (three seeds) so that all pruning methods share the recovery configuration, and report the 2×10^{-5} runs for reference. Merging the recovery adapters into the pruned weights (`merge_and_unload()`) adds the dense low-rank update BA to every adapted matrix and thereby fills the pruned entries: after merging, the WANDA models contain no zero weights (0.0% sparsity, against 10–50% before merging). We therefore record the pruning mask before merging and re-apply it afterwards, so that the evaluated models have the stated sparsity; Table 4 reports results with the re-applied mask; results without it

are not reported, because those models are dense. Structured pruning removes entire channels, so its sparsity is preserved by merging. Pruned weights are stored in dense format, so unstructured and semi-structured sparsity is not exploited by the inference kernels. For the 50% and semi-structured settings, we additionally run the *mask-aware* recovery of Section 4.2, implemented by replacing the forward pass of the LoRA layers with $\Delta W = (BA) \cdot s \odot M$ (s: the LoRA scaling); all other settings are identical to the original recovery, and merging the adapter then preserves the sparsity exactly.

5.6 Metrics

All configurations are evaluated on the 400 test samples with the original evaluation code.

Proactive prediction. Acc-P measures whether the predicted need for proactive assistance (proactive score above the threshold) matches the ground truth. The MD is the fraction of samples where assistance was needed but not predicted, and the FD is the fraction where assistance was predicted but not needed. RMSE measures the error of the predicted proactive score on the 1–5 scale.

Tool calling. Precision, recall and F1-score compare predicted and ground-truth tool names as sets. Acc-Args is described in the original paper as a per-sample measure of argument correctness (Yang et al., 2025b). The released evaluation code, which we use unchanged, computes it per tool call instead: it is one minus the share of erroneous calls over all executed calls, where a call counts as erroneous if the tool is not in the ground-truth chain or its sandbox execution returns an error, and every missed proactive sample adds one erroneous call. The argument values themselves are not compared with the reference. We report the metric as implemented, so that our numbers are comparable to the reported ones, and check the argument values separately (Section F.2).

Statistical analysis. Because all configurations are evaluated on the same 400 test samples, we compare them with a paired bootstrap over test samples (Koehn, 2004; Berg-Kirkpatrick et al., 2012) (10,000 resamples), recomputing each metric on every resample, and report the difference with its 95% confidence interval. Where several training seeds are available, each configuration's metric is averaged over its seeds within every resample. For claims of no difference we use the two one-sided tests (TOST) procedure (Schuirmann, 1987) with an equivalence margin of ± 0.03, chosen before the analysis as roughly the seed-to-seed spread plus evaluation noise; equivalence at $\alpha = 0.05$ holds if the 90% confidence interval lies inside the margin. To assess whether the ranking of methods reproduces, we compute Kendall's τ between our and the reported F1-scores. Because many comparisons are made, we also apply the Holm–Bonferroni correction (Holm, 1979) within each family of comparisons that supports one claim (for example, all quantisation comparisons) and state where an effect does not survive it. All comparisons are listed in Appendix E.

Costs. Training cost is reported as duration and peak and average GPU memory. Inference cost is reported as peak and average GPU memory (GB), average, minimum and maximum batch latency (s) and throughput (samples/s), with a batch size of 1.

6 Results

We first report the reproduction of the original results (Section 6.1) and then the efficiency experiments (Section 6.2).

6.1 Reproduction of the Original Results

Table 1 reports all methods with the released configuration, and Table 2 compares the reproduced ICL-All and *ContextAgent* results with the values reported by Yang et al. (2025b).

Table 1: Reproduction on *ContextAgentBench* (400 test samples) with the released configuration (*ContextAgent*: LoRA $r = 8$, learning rate 5×10^{-5}; mean ± standard deviation over three training seeds).

		Proactive Predictions				Tool Calling			
Model	Method	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	Vanilla ICL	0.730	0.225	0.045	2.066	0.569	0.514	0.525	0.246
	CoT	0.713	0.242	0.045	2.007	0.568	0.503	0.519	0.259
	ICL-P	0.777	0.165	0.058	2.007	0.633	0.544	0.567	0.374
	ICL-All	0.755	0.212	**0.033**	1.935	0.602	0.545	0.558	0.294
	ContextAgent	**0.915±0.007**	**0.044±0.011**	0.041±0.006	**1.124±0.033**	**0.764±0.006**	**0.745±0.006**	**0.740±0.005**	**0.593±0.008**
DeepSeek-R1-7B	Vanilla ICL	0.627	0.315	0.058	2.711	0.470	0.445	0.451	0.129
	CoT	0.632	0.302	0.065	2.645	0.471	0.450	0.455	0.153
	ICL-P	0.710	0.212	0.077	2.387	0.514	0.467	0.479	0.152
	ICL-All	0.720	0.237	**0.043**	2.393	0.515	0.488	0.494	0.173
	ContextAgent	**0.855±0.019**	**0.056±0.008**	0.089±0.016	**1.431±0.084**	**0.613±0.013**	**0.608±0.019**	**0.596±0.015**	**0.365±0.008**
Qwen2.5-7B-Ins	Vanilla ICL	0.800	0.165	0.035	2.079	0.583	0.554	0.558	0.298
	CoT	0.782	0.193	**0.025**	2.095	0.563	0.530	0.538	0.249
	ICL-P	0.880	0.072	0.048	1.726	0.568	0.546	0.548	0.283
	ICL-All	0.865	0.095	0.040	1.741	0.631	0.570	0.583	0.370
	ContextAgent	**0.912±0.010**	**0.041±0.008**	0.048±0.004	**1.129±0.052**	**0.732±0.006**	**0.716±0.010**	**0.707±0.008**	**0.546±0.016**

Table 2: Reported (Yang et al., 2025b) versus reproduced results. *Code*: released configuration (learning rate 5×10^{-5}); *paper*: learning rate 10^{-4} as stated in the original paper. Both use LoRA $r = 8$ on all linear layers. ContextAgent (ours) rows report the mean ± standard deviation over three training seeds (42, 1, 2).

		Proactive Predictions				Tool Calling			
Model	Method	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	ICL-All (reported)	0.757	0.229	0.012	1.872	0.631	0.565	0.582	0.270
	ICL-All (ours)	0.755	0.212	0.033	1.935	0.602	0.545	0.558	0.294
	ContextAgent (reported)	0.874	0.030	0.095	1.408	0.660	0.627	0.626	0.448
	ContextAgent (ours, code)	0.915±0.007	0.044±0.011	0.041±0.006	1.124±0.033	0.764±0.006	0.745±0.006	0.740±0.005	0.593±0.008
	ContextAgent (ours, paper)	0.927±0.016	0.040±0.007	0.033±0.009	1.024±0.076	0.800±0.008	0.785±0.006	0.777±0.005	0.664±0.004
DeepSeek-R1-7B	ICL-All (reported)	0.704	0.268	0.027	2.540	0.545	0.510	0.518	0.103
	ICL-All (ours)	0.720	0.237	0.043	2.393	0.515	0.488	0.494	0.173
	ContextAgent (reported)	0.888	0.027	0.085	1.319	0.676	0.648	0.647	0.468
	ContextAgent (ours, code)	0.855±0.019	0.056±0.008	0.089±0.016	1.431±0.084	0.613±0.013	0.608±0.019	0.596±0.015	0.365±0.008
	ContextAgent (ours, paper)	0.888±0.010	0.048±0.009	0.063±0.011	1.238±0.031	0.698±0.008	0.699±0.011	0.685±0.006	0.502±0.003
Qwen2.5-7B-Ins	ICL-All (reported)	0.867	0.088	0.044	1.721	0.635	0.577	0.591	0.301
	ICL-All (ours)	0.865	0.095	0.040	1.741	0.631	0.570	0.583	0.370
	ContextAgent (reported)	0.894	0.013	0.091	1.264	0.672	0.644	0.645	0.459
	ContextAgent (ours, code)	0.912±0.010	0.041±0.008	0.048±0.004	1.129±0.052	0.732±0.006	0.716±0.010	0.707±0.008	0.546±0.016
	ContextAgent (ours, paper)	0.925±0.008	0.044±0.009	0.031±0.004	1.060±0.017	0.765±0.006	0.747±0.002	0.740±0.005	0.598±0.024

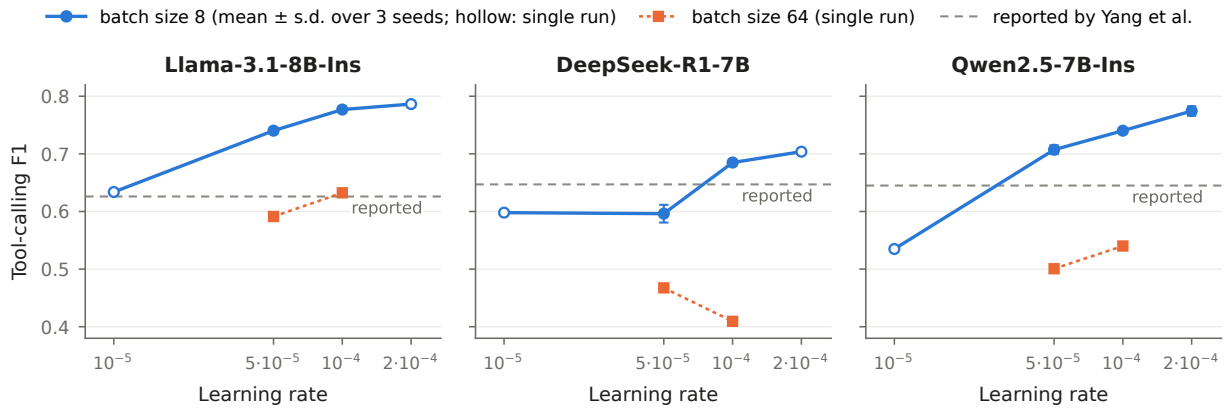


Figure 1: Tool-calling F1-score versus learning rate (LoRA $r = 8$, all linear layers). Blue: effective batch size 8 (filled: mean ± standard deviation over three seeds; hollow: single run). Orange: effective batch size 64, as with eight data-parallel GPUs (single runs). Dashed line: score reported by Yang et al. (2025b).

C1: fine-tuning outperforms in-context learning (reproduced). For all three models, *ContextAgent* achieves the highest Acc-P, the lowest MD and RMSE, and the best tool-calling scores of all methods (Table 1). The only exception is the FD, where some ICL baselines are lower, as in the original paper. For Llama and Qwen the margin over ICL-All is considerably larger than reported: tool-calling F1 improves by 0.182, 0.102 and 0.124 for Llama, DeepSeek and Qwen with the released configuration, against 0.044, 0.129 and 0.054 in the original paper.

C2: personas and thought traces help in-context learning (reproduced). ICL-All improves over Vanilla ICL in Acc-P, F1-score and Acc-Args for all three models. As in the original paper, adding thought traces alone (CoT) does not consistently help. The ranking of the five methods by tool-calling F1-score also agrees with the original paper: Kendall's τ between our and the reported values is 0.80 for each model and 0.82 over all 15 method–model pairs ($p < 0.001$).

C3: reported absolute scores (partially reproduced). The in-context baselines reproduce closely: Acc-P is within 0.02 and F1-score within 0.025 of the reported ICL-All values. All reported values are taken from Table 1 of Yang et al. (2025b); its appendix Table 6 reports different values for the Llama-3.1-8B baselines (e.g. ICL-All F1-score 0.596 instead of 0.582) with an identical *ContextAgent* row, and against those values our Llama ICL-All is 0.038 lower in F1-score. The fine-tuned models, however, differ substantially. With the released configuration, Llama and Qwen exceed the reported tool-calling F1 by 0.114 and 0.062, while DeepSeek falls 0.051 below it. With the learning rate stated in the paper, all three models exceed the reported values, by 0.151 (Llama), 0.038 (DeepSeek) and 0.095 (Qwen) in F1 and by 0.034–0.216 in Acc-Args. These are means over three training seeds; the spread over seeds (standard deviation of the F1-score at most 0.015) is small compared with these differences (Figure 1). The error profile also differs: the reported models have a low MD and a high FD (0.085–0.095), whereas our Llama and Qwen models are balanced (MD 0.040–0.044, FD 0.031–0.048); only DeepSeek at the released learning rate has a similarly high FD (0.089).

To test whether eight-GPU data-parallel training explains the gap, we trained each model with an effective batch size of 64 ($\approx$ 50 optimiser steps instead of $\approx$ 375; Section 5.3, Table 13 and the orange series in Figure 1). For Llama, batch size 64 at the paper's learning rate reproduces the reported tool-calling F1 closely (0.632 versus 0.626). For DeepSeek and Qwen, it lowers the F1-score well below the reported values (0.409 and 0.540 at 10^{-4}, against 0.647 and 0.645), which instead lie between our batch-64 and batch-8 results. Eight-GPU training is therefore a plausible explanation for the Llama result only, and since the released training script uses a single GPU, even this agreement may be coincidental.

The raw outputs show the mechanism (Section C.1): with about 50 optimiser steps at the released learning rate, DeepSeek never adopts the trained output format (none of its 400 outputs contains a proactive score) and Qwen omits the score in 48% of its outputs, whereas Llama learns the format at both learning rates; even at 10^{-4}, the decisions of DeepSeek and Qwen remain clearly worse than with 375 optimiser steps. In either case, the qualitative conclusions of the original paper hold.

Cost of the reproduced models. The reproduced models peak at about 21 GB of GPU memory in training and train in 45–58 minutes (Table 9); at inference they need 14.8–15.6 GB and 8.3–9.0 s per sample at batch size 1 (Table 14). These are the reference costs for the efficiency experiments.

6.1.1 Lite and Out-of-Domain Claims

Beyond its Table 1, Yang et al. (2025b) report results on *ContextAgentBench-Lite* (their Table 2), on an out-of-domain split by scenario (their Section 5.4 and Table 9), with missing sensors (their Table 3) and against proprietary models (their Table 4). The release does not contain the train/test split of *ContextAgentBench-Lite* that its code expects, nor the out-of-domain split. Both are recoverable from the history of the authors' repository: a Lite split of 196 training and 84 test samples, used for Lite-specific fine-tuning on the perception-pipeline text, and an out-of-domain split that holds out the scenarios Travel, Others and Cooking (289 samples), set explicitly in the split script although the paper describes the split as random. Neither the 84 nor the 300 released Lite samples match the resolution of the reported Lite scores: 120 of the 123 proportions in their Table 7 are multiples of 0.01, consistent with about 100 evaluated samples. We therefore

evaluate (i) the original protocols on the recovered splits, with the released data, and (ii) the transfer of our *ContextAgentBench* models to the Lite input and a three-fold scenario split, reporting the 119 Lite samples that are not in the training split separately from the 181 that are.

Lite/OOD results (runs: `runs_lite.txt`, `runs_ood.txt`, ICL on Lite, `runs_ood_orig.txt`, `runs_lite_orig.txt`): one table (fine-tuned vs ICL; Lite: original protocol on the 84 test ids, transfer on unseen 119 vs seen 181; OOD: original split vs reported Table 9, three-fold split as difference-in-differences against the full-data model) and 3–4 sentences. Note for the ICL-Lite analysis: three of the ten ICL demonstrations are Lite samples (all in the seen subset).

6.2 Efficiency under Resource Constraints

All experiments in this section use the *ContextAgent* configuration with thought traces and personas. Unless stated otherwise, the LoRA configuration is the released default: rank $r = 8$, all linear layers, learning rate 5×10^{-5}. Because all sweeps are evaluated on the test split (Section 5.3), they are a sensitivity analysis rather than a model selection procedure.

6.2.1 LoRA Configuration

The learning rate is the dominant LoRA setting (Figure 1; all values in Table 17, Appendix C). Above 5×10^{-5}, tool-calling performance increases with the learning rate for all three models. Moving from the released 5×10^{-5} to the paper's 10^{-4} raises the F1-score from 0.740 to 0.777 (Llama), from 0.596 to 0.685 (DeepSeek) and from 0.707 to 0.740 (Qwen), averaged over three seeds; all three differences are significant (+0.037, +0.089 and +0.033) and survive the Holm correction, and the seed standard deviation is at most 0.015. At 2×10^{-4} all three models improve further (Llama 0.786 and DeepSeek 0.704, single runs; Qwen 0.774 ± 0.008, three seeds), and selection on a held-out validation split also picks 2×10^{-4} for all three (see below). 10^{-5} is clearly worse for Llama and Qwen (0.634 and 0.535), whereas for DeepSeek it is on par with the released value (0.598). At the released learning rate, a higher rank partly compensates for the low learning rate (Llama 0.735 at $r = 8$ versus 0.782 at $r = 16$; Qwen up to 0.770 at $r = 32$; DeepSeek up to 0.690 at $r = 64$; single runs, Table 15), and adapting all linear layers is the best or on par with adapting only the MLP layers, whereas attention-only adaptation is the worst for every model, by 0.04–0.10 in F1-score (Table 16). Once the learning rate is 10^{-4}, the rank no longer matters (Section 6.2.3). The released learning rate is therefore not anomalous but lies on the lower side of an increasing trend, consistent with the known sensitivity of LoRA to the learning rate (Biderman et al., 2024) and with a training set of only 600 samples.

Validation-based selection. Because the released split has no validation set, the sweeps above are evaluated on the test split. To check whether this selection inflates our results, we split the 600 training samples into 500 for training and a validation set of 100, stratified by scenario and proactive label, and repeated the configuration grid on it (ranks 8, 32 and 64; learning rates 5×10^{-5}, 10^{-4} and 2×10^{-4}). Validation selects 2×10^{-4} for all three models (Llama and DeepSeek with $r = 32$, Qwen with $r = 8$). Retrained on all 600 samples with three seeds, the validation-selected configurations reach test F1-scores of 0.795 ± 0.008 (Llama), 0.702 ± 0.010 (DeepSeek) and 0.774 ± 0.008 (Qwen). They are at least as good as the test-chosen combined configurations of Section 6.2.3 (+0.025, +0.004 and +0.021; the Llama gain survives the Holm correction, the Qwen gain does not, and DeepSeek is equivalent within ± 0.03) and as the paper's configuration (+0.018, +0.017 and +0.034; significant only for Qwen; Table 26). Test-set selection therefore did not make our configurations optimistic; if anything, the learning-rate grid stopped too early.

6.2.2 Quantisation

At the released configuration (single runs; Table 18, Appendix C), 8-bit LLM.int8 changes the tool-calling F1-score by +0.006 to +0.012 and 4-bit NF4 by −0.013 to −0.028 relative to 16-bit. None of these differences is significant, and the only nominally significant effect, a lower Acc-P for 4-bit Qwen (−0.052, 95% CI $[-0.090, -0.015]$), does not survive the Holm correction over the quantisation comparisons. Single runs are not precise enough to show equivalence within ± 0.03, so we report no detectable difference rather than equivalence; this agrees with what is known for QLoRA on other tasks (Dettmers et al., 2023). The costs differ more than the scores (Table 19): for Llama, peak inference memory falls from 15.6 GB to 9.3 GB at 8-bit and 8.2 GB at 4-bit, but with the bitsandbytes kernels LLM.int8 roughly doubles the time per sample,

and NF4 is 17–42% slower than 16-bit. In training, 4-bit quantisation reduces peak memory by 10–47% depending on the model, at 9–29% longer training time (Table 10).

6.2.3 Combined Configuration

Table 3: Best configuration per model found on the test split (optimistic; see Section 5.3), at 16-bit and 4-bit. Mean ± standard deviation over three training seeds (42, 1, 2). For Qwen at 16-bit, seed 42 was evaluated a second time because the first prediction file was incomplete (340 of 400 samples).

		Proactive Predictions				Tool Calling			
Model	Config	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
LLaMA-3.1-8B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	0.928±0.003	**0.033±0.007**	0.039±0.009	0.998±0.014	0.788±0.005	0.784±0.017	0.771±0.011	0.657±0.019
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	**0.933±0.010**	0.038±0.005	**0.028±0.006**	**0.981±0.090**	**0.807±0.016**	**0.795±0.009**	**0.787±0.012**	**0.664±0.018**
DeepSeek-R1-7B	$r = 64, lr = 1.0 \times 10^{-4}$, all, 16-bit	**0.892±0.016**	**0.044±0.005**	0.064±0.016	**1.225±0.074**	**0.717±0.020**	0.712±0.024	0.698±0.022	0.516±0.020
	$r = 64, lr = 1.0 \times 10^{-4}$, all, 4-bit	0.888±0.011	0.049±0.010	**0.062±0.005**	1.259±0.042	0.716±0.011	**0.729±0.018**	**0.708±0.012**	**0.535±0.014**
Qwen2.5-7B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	0.925±0.010	**0.031±0.003**	0.044±0.008	1.084±0.062	0.777±0.011	0.761±0.018	0.753±0.014	0.624±0.028
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	**0.928±0.006**	0.039±0.006	**0.033±0.005**	**1.042±0.030**	**0.788±0.005**	**0.776±0.008**	**0.769±0.004**	**0.648±0.011**

Table 3 combines the best settings found above for each model. For Llama and Qwen this is $r = 32$, learning rate 10^{-4} and all linear layers; for DeepSeek it is $r = 64$, learning rate 10^{-4} and all linear layers, the best DeepSeek configuration in our sweeps. All six rows are averaged over three training seeds. Once the learning rate is 10^{-4}, the higher rank adds no significant gain for any model: compared with $r = 8$ at 10^{-4} (three seeds each), the F1-score changes by -0.006 for Llama (0.771 versus 0.777), $+0.013$ for DeepSeek (0.698 versus 0.685) and $+0.013$ for Qwen (0.753 versus 0.740), and all 95% confidence intervals include zero (Table 26). The improvement over the released default therefore comes from the learning rate. The 4-bit version of each combined configuration scores slightly higher than the 16-bit version (Llama 0.787 versus 0.771, DeepSeek 0.708 versus 0.698, Qwen 0.769 versus 0.753), but none of these differences is significant (Table 26). With three seeds, the evaluation is precise enough to show equivalence within ±0.03 for DeepSeek's F1-score and for Acc-P of all three models. 4-bit models with a suitable configuration therefore match or exceed the released 16-bit configuration (0.740, 0.596 and 0.707) at about half the inference memory.

At inference, 4-bit quantisation of the combined configurations reduces peak GPU memory by 47–48% for all three models (Llama 15.6 to 8.3 GB, DeepSeek 14.8 to 7.8 GB, Qwen 14.8 to 7.7 GB), with 53% (Llama) and 33% (Qwen) more time per sample and almost unchanged latency for DeepSeek (Table 20); training costs are in Table 11.

6.2.4 Pruning

Pruning experiments use `Llama-3.1-8B-Instruct` with the released configuration; all results are after LoRA-based recovery fine-tuning. The dense reference is the seed-42 *ContextAgent* run (the $r = 8$ row of Table 15; F1-score 0.735, Acc-Args 0.595).

Table 4: Unstructured and structured pruning of `Llama-3.1-8B-Instruct` after recovery fine-tuning. For unstructured pruning, the binary pruning mask is re-applied after merging the recovery adapter, ensuring the evaluated weights have the exact stated sparsity; rows with three evaluations report the mean. ‡Mask-aware recovery: the LoRA update is restricted to the unpruned weights (Section 5.5). Rows marked 10^{-4} use mask-aware recovery with learning rate 10^{-4} and the stated rank instead of the released recovery configuration ($r = 8$, 5×10^{-5}); the $r = 32$ row is the mean ± standard deviation over three seeds, all other mask-aware rows are single runs. Dense reference at 10^{-4}, $r = 8$: F1-score 0.777 ± 0.005, Acc-Args 0.664 ± 0.004 (three seeds). Dense reference: F1-score 0.735, Acc-Args 0.595 (seed-42 run, $r = 8$ row of Table 15).

			Proactive Predictions				Tool Calling			
Structure	Pruning	%	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Unstructured	Wanda	10	0.897	0.033	0.070	1.188	0.752	0.710	0.715	0.598
		30	0.920	0.048	0.033	1.108	0.764	0.692	0.708	0.603
		50	0.886	0.067	0.048	1.276	0.693	0.618	0.634	0.459
		50‡	0.895	0.072	0.033	1.217	0.724	0.718	0.706	0.521
		50‡, 10^{-4}, r=8	0.912	0.058	**0.030**	1.124	0.741	0.725	0.717	0.566
		50‡, 10^{-4}, r=32	**0.932±0.003**	0.034±0.004	0.034±0.001	**0.988±0.070**	**0.789±0.002**	**0.781±0.005**	**0.770±0.002**	**0.636±0.005**
	Magnitude	10	0.915	0.043	0.043	1.134	0.740	0.699	0.705	0.577
		30	0.907	**0.030**	0.062	1.180	0.727	0.658	0.673	0.577
		50	0.515	0.480	0.005	4.629	0.449	0.445	0.446	0.021
		50‡	0.887	0.050	0.062	1.279	0.697	0.695	0.679	0.481
Structured	Activation	10	**0.870**	**0.083**	**0.048**	**1.371**	**0.652**	**0.651**	**0.637**	**0.410**
		30	0.718	0.172	0.110	1.944	0.493	0.490	0.481	0.236
		50	0.522	0.315	0.163	2.516	0.315	0.309	0.310	0.047
	Magnitude	10	0.470	0.530	0.000	3.407	0.468	0.468	0.468	0.000
		30	0.470	0.530	0.000	3.407	0.468	0.468	0.468	0.000
		50	0.470	0.530	0.000	3.407	0.468	0.468	0.468	0.000

Table 4 shows that unstructured pruning preserves performance much better than structured pruning. With the binary mask re-applied after merging the recovery adapter, WANDA preserves performance well at 10% and 30% sparsity: the F1-score remains at 0.715 and 0.708 (dense reference: 0.735), and Acc-Args is maintained at 0.598 and 0.603 (dense: 0.595). At 50% sparsity, however, enforcing strict zeros degrades performance noticeably, with the F1-score dropping by 0.10 to 0.634 and Acc-Args to 0.459. Magnitude pruning, which ignores activations, degrades faster. At 10% sparsity it is still close to the dense model (F1-score 0.705; difference -0.030, 95% CI $[-0.063, 0.002]$), but at 30% it already loses 0.06 (0.673, CI $[-0.100, -0.024]$), and at 50% the model largely loses the output format: 90% of its outputs contain no proactive score, so Acc-P falls to 0.515, MD rises to 0.480 and Acc-Args to 0.021. At the same 50% sparsity, WANDA keeps an F1-score of 0.634, which confirms that activation-aware importance matters at high sparsity (Sun et al., 2024).

Most of this loss is caused by the recovery procedure rather than by the sparsity itself. An unconstrained recovery update is spread evenly over pruned and kept weights, so re-applying the mask discards a share of it equal to the sparsity (49.7% of its squared norm at 50% WANDA sparsity; Table 25). With mask-aware recovery, which trains the update only on the unpruned weights, every setting improves significantly (Table 26): WANDA at 50% rises from 0.634 to 0.706 and is no longer significantly below the dense model in F1-score, although its Acc-Args remains lower (0.521 versus 0.595), and magnitude pruning at 50% rises from 0.446 to 0.679, because its outputs regain the format (outputs without a parseable proactive score: 92.5% to 0.5%; Section 6.3). The price is a three times longer recovery training (about 8,480 s instead of 2,770 s), because the masked update is materialised as a full matrix in every forward pass; inference cost is unchanged. The released recovery configuration ($r = 8$, 5×10^{-5}) is itself sub-optimal for the dense model, and with the paper's learning rate mask-aware recovery closes most of the remaining gap. With $r = 32$ and three seeds, WANDA at 50% reaches 0.770 ± 0.002, against 0.777 ± 0.005 for the dense model trained at the same learning rate and 0.771 ± 0.011 for the combined configuration; both differences lie within the equivalence margin of ±0.03 (TOST), as does the difference in Acc-P. Rank alone makes no significant difference for the dense model at 10^{-4} (Section 6.2.3), so this is not a rank effect. The semi-structured patterns remain below the dense model: 4:8 reaches 0.756 ± 0.008, with no significant difference in F1-score but a lower Acc-Args, and 2:4 reaches 0.719 ± 0.022 (-0.058, 95% CI $[-0.081, -0.034]$), with the largest seed spread of all configurations (0.704–0.744). Single runs with $r = 8$ reach 0.717 (50%), 0.745 (4:8) and 0.711 (2:4), significantly above mask-aware recovery at 5×10^{-5} for the two semi-structured patterns but not for

50% (all comparisons in Table 26). At 50% unstructured sparsity, a pruned model with mask-aware recovery at the paper's learning rate therefore performs on par with the dense model fine-tuned in the same way.

Structured activation-based pruning loses 0.10 in F1-score already at 10% (0.637) and collapses at 50% (0.310). Structured magnitude pruning yields identical results at all three ratios, with MD = 0.530 and FD = 0.000: none of its outputs contains a parseable proactive score, so the evaluation treats every sample as a decision not to act. Its Acc-P of 0.470 and F1-score of 0.468 therefore correspond to the share of non-proactive samples, whose empty tool set is trivially matched. This is a degenerate solution rather than a gradual degradation.

Training costs (Table 12, Appendix B) show that unstructured pruning does not reduce recovery training time or memory, whereas 50% structured pruning reduces training time by about 48% and peak memory by 43% compared to 10% structured pruning.

At inference (Table 21, Appendix C), unstructured and semi-structured pruning leave memory and latency unchanged, because the pruned weights are stored and multiplied densely; this also holds after mask-aware recovery. Executing the 2:4 models with PyTorch's sparse tensor-core kernels leaves the task metrics unchanged but makes generation about 2.8 times slower at batch size one (Appendix D). Structured magnitude pruning is 3.5–5 times *slower* than the dense model despite its smaller size, because its degenerate outputs run until the limit of 1,024 new tokens.

6.2.5 Small Base Models

We fine-tune 1–1.5B models of the same three families directly at the full sequence length of 2048 tokens, with the configuration that works best for the 7–8B models (LoRA on all linear layers, learning rate 10^{-4}, five epochs): `Llama-3.2-1B-Instruct`, `DeepSeek-R1-Distill-Qwen-1.5B` and `Qwen2.5-1.5B-Instruct`, each with $r = 8$ and $r = 32$ ($r = 32$ with three seeds), and `Qwen2.5-1.5B` additionally with a 4-bit NF4 base model.

Table 5: 1–1.5B models fine-tuned at the full sequence length of 2048 tokens (LoRA on all linear layers, learning rate 10^{-4}, five epochs; $r = 8$: single runs; $r = 32$, including the 4-bit NF4 base model: mean ± standard deviation over three seeds). Inference cost at batch size 1 on the L4 GPU, averaged over the runs; the time per sample varies between runs of the same configuration (e.g. 3.7–4.7 s for Qwen2.5-1.5B), while the memory is identical.

		Proactive Predictions				Tool Calling				Inference	
Model	Rank	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑	GPU (GB)↓	s/sample↓
Llama-3.2-1B-Ins	8	0.820	0.105	0.075	1.576	0.636	0.645	0.627	0.417	3.3	2.5
	32	0.875 ± 0.005	0.074 ± 0.008	0.051 ± 0.004	1.337 ± 0.041	0.686 ± 0.007	0.688 ± 0.005	0.673 ± 0.007	0.468 ± 0.013	3.3	3.4
DeepSeek-R1-1.5B	8	0.835	0.060	0.105	1.550	0.524	0.545	0.525	0.267	3.8	3.8
	32	0.861 ± 0.009	0.067 ± 0.006	0.073 ± 0.004	1.424 ± 0.060	0.611 ± 0.011	0.619 ± 0.006	0.603 ± 0.009	0.385 ± 0.012	3.9	4.3
Qwen2.5-1.5B-Ins	8	0.858	0.095	0.048	1.411	0.669	0.664	0.653	0.441	3.8	3.5
	32	0.903 ± 0.019	0.058 ± 0.009	**0.039 ± 0.011**	1.214 ± 0.093	0.724 ± 0.006	0.722 ± 0.012	0.707 ± 0.006	0.528 ± 0.024	3.8	4.2
	32, 4-bit	**0.907 ± 0.008**	**0.052 ± 0.006**	0.042 ± 0.009	**1.148 ± 0.045**	**0.726 ± 0.012**	**0.724 ± 0.014**	**0.709 ± 0.012**	**0.543 ± 0.017**	2.2	12.1

All three small models learn the output format: at most 0.3% of their outputs lack a parseable proactive score (Section 6.3). The higher rank helps every model, by 0.05–0.08 in F1-score (Llama-3.2-1B 0.627 to 0.673 ± 0.007, DeepSeek-R1-1.5B 0.525 to 0.603 ± 0.009, Qwen2.5-1.5B 0.653 to 0.707 ± 0.006), far more than the seed variation (standard deviation at most 0.009). Qwen2.5-1.5B with $r = 32$ is equivalent to Qwen2.5-7B with the released configuration (F1-score 0.707 for both; TOST within ±0.03) at 3.8 GB instead of 14.8 GB of inference memory and 4.2 s instead of 8.3 s per sample. Against the 7–8B models trained with the same learning rate, however, all three small models are significantly worse: by 0.034 in F1-score for Qwen ($[-0.057, -0.011]$; Acc-Args -0.071), 0.082 for DeepSeek and 0.104 for Llama. With a 4-bit base model, Qwen2.5-1.5B keeps its F1-score (0.709 ± 0.012 over three seeds, equivalent to the 16-bit runs within ±0.03) at only 2.2 GB, but on-the-fly dequantisation raises the time per sample to 12.1 s. The rank was chosen on the test split, so these results are optimistic in the same way as the other configuration sweeps.

Distillation from a fine-tuned teacher. To test whether a teacher adds anything beyond direct fine-tuning, we used sequence-level distillation (Kim & Rush, 2016) at the full sequence length: each fine-tuned

7–8B model ($r = 8$, 10^{-4}) generated outputs for the 600 training samples, and the 1–1.5B student of the same family was fine-tuned on these generations instead of the reference outputs, with the same recipe as the direct fine-tuning ($r = 32$, 10^{-4}, 2,048 tokens, three seeds). Because the teachers reproduce the reference tool calls on their own training data only partly (F1-score 0.72–0.81), we also trained on a filtered set that keeps a teacher output only where its proactive decision and tool selection match the reference and uses the reference output otherwise (57–66% teacher outputs). Only the weakest student shows any gain (Table 6): Llama-3.2-1B improves in Acc-P (+0.026) and Acc-Args (+0.036) with both variants, and with filtering also in F1-score (+0.021, 95% CI $[0.001, 0.041]$), but none of these gains survives the Holm correction over the distillation comparisons. For DeepSeek-R1-1.5B and Qwen2.5-1.5B, neither variant differs significantly from direct fine-tuning, and filtering never differs significantly from unfiltered distillation. At this data size, direct fine-tuning at full length therefore captures what a teacher's outputs can transfer, and the gap between the 1–1.5B and the 7–8B models remains.

Table 6: Sequence-level distillation from the fine-tuned 7–8B teacher into the 1–1.5B student of the same family ($r = 32$, 10^{-4}, 2,048 tokens), against direct fine-tuning on the reference outputs. Tool-calling F1-score, mean ± standard deviation over three seeds; Δ: difference of filtered distillation to direct fine-tuning with 95% paired bootstrap CI (none significant after Holm correction). Filtered: teacher output only where it matches the reference decision and tools. Teacher: F1-score of the teacher ($r = 8$, 10^{-4}, seed 1) on the test set.

Student	Direct	KD	KD, filtered	ΔF1 (filtered)	Teacher
Llama-3.2-1B	0.673 ± 0.007	0.691 ± 0.005	0.694 ± 0.017	+0.021 [0.001, 0.041]	0.772
DeepSeek-R1-1.5B	0.603 ± 0.009	0.602 ± 0.025	0.614 ± 0.018	+0.011 [−0.015, 0.037]	0.688
Qwen2.5-1.5B	0.707 ± 0.006	0.693 ± 0.007	0.709 ± 0.012	+0.003 [−0.019, 0.025]	0.737

Logit-level distillation at full length (teacher top-32 logits cached, `kd_logit.py`; 3 students × 3 seeds) is running: add a column to Table 6 and one sentence.

6.2.6 Decision-First Output Order

Proactive agents spend most of their time deciding *not* to act, yet the *ContextAgent* output format places the proactive score after the thought trace, so every negative decision costs a full reasoning trace. We therefore fine-tuned Llama-3.1-8B ($r = 8$, 10^{-4}, three seeds) on training targets in which the proactive-prediction block precedes the thought trace. The models learn the order (all test outputs start with the score) and predict that no assistance is needed for 178–187 of the 400 samples; on these, they reach the score after 22 tokens instead of the 89 they generate in total. Stopping there would remove 22–23% of all generated tokens and, at the measured 66 ms per token, about 2.0 s per sample (from 9.6 to about 7.6 s; without early stopping, the score-first models are slightly slower than the thought-first models, 9.6 against 9.0 s). The proactive decision does not suffer: Acc-P is 0.925 ± 0.018 against 0.927 ± 0.016 (equivalent within ± 0.03). Tool calling loses slightly: the F1-score is 0.758 ± 0.014 against 0.777 ± 0.004 (not significant), and Acc-Args drops from 0.664 to 0.626 (-0.038, 95% CI $[-0.067, -0.009]$; Table 26). The lower Acc-Args comes from tool selection: the right tool is found as often as with the thought-first order, and its argument values are as close to the reference (token-level F1-score 0.592 versus 0.582; Section F.2), but tool-calling precision is lower (-0.024, 95% CI $[-0.046, -0.001]$, while recall does not differ significantly), so more superfluous tools are called. Emitting the decision first therefore preserves the decision to act and saves about a fifth of the latency, at a small cost in tool selection.

Decision-first output order: replace the per-token estimate by the measured early exit on vLLM (job 913085) when it finishes.

6.3 Where the Models Fail

To show *how* the configurations fail, we assign every test sample to exactly one stage of the decision pipeline: (a) no parseable proactive score; (b) a parseable but wrong proactive decision; (c) a correct decision that no assistance is needed; (d) a correct proactive trigger with a wrong or incomplete tool set; (e) a correct trigger

and the exact tool set, but at least one wrong tool argument; and (f) a fully correct response. Table 7 shows representative configurations; the full cascade for all configurations is in Table 27 (Appendix F.1).

Table 7: Error cascade for representative configurations (% of the 400 test samples; each sample is counted in exactly one stage; stages as in the text). Rows with three runs: mean over seeds or repeated evaluations. ‡Mask-aware recovery. Full table: Table 27.

Model	Configuration	Runs	(a)	(b)	(c)	(d)	(e)	(f)
Llama-3.1-8B-Ins	LoRA 10^{-4} (paper)	3	0.0	7.2	43.8	32.0	0.0	17.0
DeepSeek-R1-7B	ICL-All	1	55.5	8.8	6.5	27.8	0.5	1.0
Qwen2.5-7B-Ins	4-bit, 5×10^{-5}	1	20.0	8.2	27.8	32.8	0.0	11.2
Llama-3.1-8B-Ins (pruned)	Magnitude 50%, re-masked	1	92.5	1.5	1.2	4.8	0.0	0.0
	Magnitude 50%‡	1	0.5	11.0	40.5	37.0	0.0	11.0
	Wanda 2:4, re-masked	3	34.0	15.7	13.5	33.1	0.9	2.8
	Wanda 2:4‡	1	0.0	12.5	39.0	39.8	0.2	8.5
	Wanda 50%‡, 10^{-4}, r=32	3	0.1	6.8	43.5	33.3	0.0	16.2
	Structured activation 50%	1	0.5	47.5	30.5	21.5	0.0	0.0
	Structured magnitude 50%	1	100	0	0	0	0	0
Qwen2.5-1.5B-Ins	LoRA r=32, 2048 tokens	3	0.0	9.7	43.1	35.9	0.0	11.3

Three patterns emerge. First, fine-tuned models fail almost only on *tool selection* (stage (d)); stage (e) stays at or below 0.1% for every fine-tuned 16-bit 7–8B model, so the differences in Acc-Args come from choosing the wrong tools rather than from calls that fail in the sandbox (the only argument check of the released metric). Checked directly, the arguments of correctly chosen tools are almost always sensible: in a blind sample of 150 calls, 84% were judged correct and 94% at least partially correct, although only 16% match the reference string exactly (Section F.2). Second, *format failures* (stage (a)) mark the configurations that break: the in-context baselines (55.5% for DeepSeek), 4-bit Qwen at the released learning rate (20%), and, with re-masked recovery, 50% magnitude pruning (92.5%) and Wanda 2:4 (34%). Mask-aware recovery removes these failures (0.5% and 0%), so the collapse is caused by the recovery procedure rather than by the sparsity, and at the paper's learning rate it comes close to the dense model's share of fully correct responses (16.2% versus 17.0%, three seeds each). Third, structured pruning fails differently: activation-based pruning mainly causes *wrong proactive decisions* (47.5% at 50%), and magnitude-based pruning loses the format completely. The 1–1.5B models behave like the 7–8B models, losing mainly on tool selection.

7 Discussion

7.1 What Reproduces and What Does Not

The qualitative claims of *ContextAgent* hold: fine-tuning with persona context and thought traces outperforms in-context learning for all three base models (C1), and personas plus thought traces improve in-context learning (C2). The in-context baselines also reproduce numerically. The fine-tuned results do not: with the released code, two of three models score clearly higher than reported, and with the learning rate stated in the paper all three do (C3).

Claim by claim. Table 8 summarises the reproduction status of every claim of the original paper and the documentation gaps behind the differences.

Two under-specified details make this difference hard to attribute. First, the paper and the released configuration disagree on the learning rate (10^{-4} versus 5×10^{-5}), and the choice between these two values alone changes tool-calling F1 by 0.03–0.09 for the same model (Table 17). Second, the paper reports eight GPUs, but not whether training was data-parallel or what the resulting effective batch size was. With only 600

Table 8: Reproduction status of the claims of Yang et al. (2025b) (top) and the documentation gaps we found in the paper and its release (bottom).

Claim (original paper)	**Status**	**Evidence**
C1: fine-tuning with personas and thought traces outperforms all in-context baselines (Table 1)	Reproduced	All three models, all metrics except FD (Table 1)
C2: personas and thought traces improve in-context learning (Table 1)	Reproduced	All three models; method ranking agrees (Kendall's $\tau = 0.82$)
C3a: absolute scores of the in-context baselines	Reproduced	ICL-All within 0.02 Acc-P and 0.025 F1 (Table 2)
C3b: absolute scores of the fine-tuned models	Not reproduced	Depend on the learning rate: above the reported values for all models at 10^{-4}, mixed at the released 5×10^{-5} (Table 2, Figure 1)
Results on *ContextAgentBench-Lite* (Tables 2 and 7)	Pending	Split not released (recovered from repository history); 181 of the 300 Lite samples are in the training split (Section 6.1.1)
Out-of-domain generalisation (Section 5.4, Table 9)	Pending	Split not released and set manually, not randomly (Section 6.1.1)
Robustness to missing sensors (Table 3)	Not re-tested	No code released; only 32 Lite samples carry audio input
Comparison with proprietary models (Table 4)	Out of scope	Paid, changing APIs
Documentation gap	**Effect**	**Where**
Learning rate: 10^{-4} in the paper, 5×10^{-5} in the released configuration	Up to 0.09 F1	Table 2, Figure 1
Eight GPUs reported, single-GPU training script released; effective batch size not stated	Batch 64 matches the reported Llama score only	Table 13
No DeepSeek-R1-7B training configuration released	Configuration had to be derived	Section 5.3
No seeds or run-to-run variation reported	Uncertainty of reported scores unknown	Section 5.6
Acc-Args described per sample, implemented per tool call without comparing argument values	Metric measures tool choice and execution, not argument values	Section 5.6, Section F.2

training samples, an eight-fold difference in batch size changes the number of optimiser steps from about 375 to about 50, which may leave the original models undertrained. Our batch-64 runs support this for Llama, whose reported scores they reproduce (0.632 versus 0.626), but not for DeepSeek and Qwen, which fall well below their reported values at batch size 64 (Table 13); for these two models the cause of the difference remains open. Since the released training script runs on a single GPU, the agreement for Llama may also be coincidental. Our recommendations for future users follow in Section 7.3.

A practical consequence is that the efficiency results below are measured against a stronger baseline than the paper reports: an efficiency technique that "preserves the reported performance" may still lose a substantial part of the performance the released code can reach.

What was easy and what was difficult. The authors' release made the reproduction largely straightforward: the code, the data split, the LLaMA-Factory training configurations and the evaluation scripts

are public, and the strongest in-context baseline (ICL-All) reproduced within 0.02 Acc-P and 0.025 F1 of the reported values without any changes. Training and evaluating one configuration fits on a single 24 GB GPU. The difficulties lay in what the release and the paper leave unspecified or contradict: the learning rate differs between paper and code; the hardware is described (eight GPUs) but not the parallelism, so the effective batch size and number of optimiser steps had to be inferred; and the paper reports neither seeds nor run-to-run variation, so the uncertainty of the reported scores is unknown. We contacted the original authors to clarify the learning rate and the training setup; at the time of submission we had not received a reply.

Update this sentence if the original authors reply before submission (contacted 13 Oct 2026).

7.2 Efficiency of the Proactive Reasoner

The learning rate matters more than the adapter. Within the LoRA design space, the learning rate has by far the largest effect: moving from the released to the paper's value improves every model significantly, and validation-based selection picks an even larger value. Rank and target modules matter only at the low released learning rate, where a higher rank partly compensates; at 10^{-4}, rank 8 on all linear layers is sufficient for all three models (Section 6.2.3). Efficiency claims that rest on a smaller adapter or fewer adapted modules should therefore be made only after the learning rate has been tuned.

Quantisation. 4-bit NF4 quantisation of the base model halves inference memory without a detectable loss in task performance, at the released configuration (single runs) and at the combined configurations, where three seeds show equivalence within ± 0.03 for Acc-P of all three models and for the F1-score of DeepSeek. With the bitsandbytes runtime, this memory saving costs up to 53% more time per sample, so the choice between 16-bit and 4-bit depends on whether memory or latency is the binding constraint.

Pruning: keep the mask through recovery. With the pruning mask preserved, unstructured Wanda pruning keeps the task metrics up to 30% sparsity. At 50% sparsity and with semi-structured patterns, the result depends on the recovery step more than on the pruning criterion. A recovery adapter trained without the mask and re-masked after merging discards a share of its update equal to the sparsity (Table 25) and loses 0.10 F1 at 50% sparsity and 0.12 and 0.27 F1 with the 4:8 and 2:4 patterns; restricting the update to the unpruned weights reduces these losses to 0.03 (not significant), 0.07 and 0.09, and with the paper's learning rate and $r = 32$ brings Wanda at 50% to the dense model trained in the same way (F1-score 0.770 versus 0.777, three seeds each, equivalent within ± 0.03). Sparsity-preserving fine-tuning has been proposed before (Lu et al., 2024; Muñoz et al., 2024); our results show that in a standard merge-and-re-mask pipeline, the recovery step alone can account for most of the apparent cost of pruning, and for the format collapse of magnitude and 2:4 pruning. The preserved accuracy comes without any saving in our setting, however: the pruned weights are stored densely, and the 2:4 sparse kernels made generation slower at batch size one. Structured pruning of MLP dimensions does reduce memory and latency, but loses 0.10 F1 already at 10%, and with magnitude-based importance it produces no parseable output at all.

Smaller base models. Fine-tuning a 1–1.5B model directly at the full sequence length is the most effective way to reduce memory and latency, but it is not free. Qwen2.5-1.5B is equivalent to Qwen2.5-7B with the released configuration (F1-score 0.707 versus 0.707, equivalent within ± 0.03) at a quarter of the inference memory and half the latency, but it stays significantly below Qwen2.5-7B trained with the same learning rate (-0.034 in F1-score, 95% CI $[-0.057, -0.011]$; -0.071 in Acc-Args). The gap is larger for Llama-3.2-1B (-0.104) and DeepSeek-R1-1.5B (-0.082) against their 7–8B counterparts. Distillation from the fine-tuned 7–8B models does not close this gap (Table 6). At a similar memory budget, however, structured pruning of Llama-3.1-8B (50%, 5.7 GB) reaches an F1-score of only 0.310, so when memory must fall well below 8 GB, a smaller base model clearly dominates structured pruning of a larger one.

Cost–performance trade-off. Figure 2 places all configurations in the space of task performance, memory and latency. Among the 7–8B models, 4-bit quantisation of a well-configured model offers the best trade-off: it matches or exceeds the released 16-bit model at about half the inference memory. Below 8 GB, only the 1–

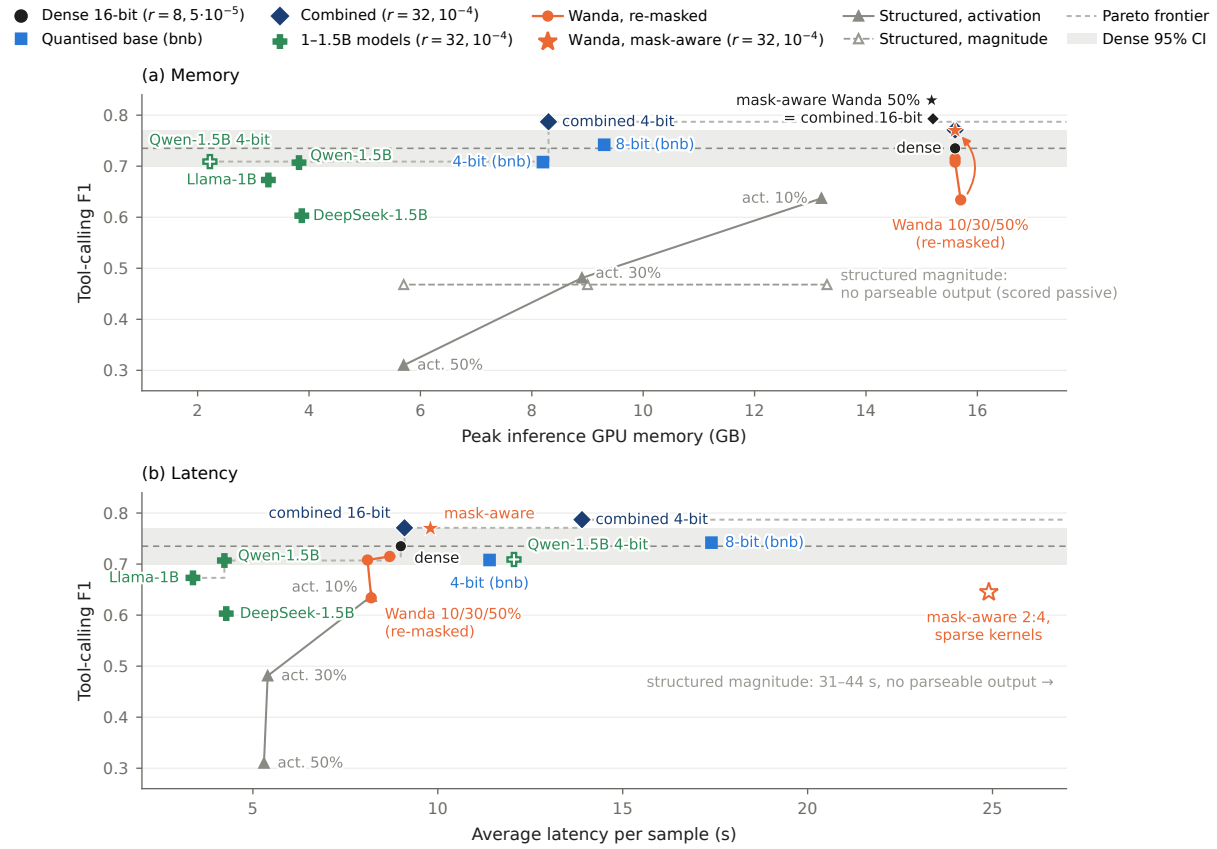


Figure 2: **Cost–performance trade-off** on *ContextAgentBench*: tool-calling F1-score against (a) peak inference GPU memory and (b) average latency per sample at batch size 1. All compression techniques are applied to `Llama-3.1-8B-Instruct`; the green crosses are 1–1.5B models of other families fine-tuned directly at the full sequence length ($r = 32$, learning rate 10^{-4}, mean over three seeds; hollow: 4-bit base; Table 5). The dashed line and grey band mark the dense 16-bit model ($r = 8$, 5×10^{-5}, seed 42) and its 95% bootstrap CI; "combined" uses $r = 32$ and 10^{-4} (mean over three seeds). Pruned weights are stored densely, so unstructured WANDA pruning stays at the dense model's memory; with mask-aware recovery ($\star$, $r = 32$, 10^{-4}, mean over three seeds) Wanda at 50% reaches the combined 16-bit model (the two points coincide in (a)), while executing the mask-aware 2:4 model with sparse kernels (hollow star) slows decoding to about 25 s per sample. Structured magnitude pruning produces no parseable proactive score and is therefore scored as never acting (F1 0.468, the share of samples that need no assistance) and generates up to the 1,024-token limit, so it lies outside panel (b) at 31–44 s per sample. The dotted step line is the Pareto frontier; points towards the upper left are better.

1.5B models keep a usable F1-score, Qwen2.5-1.5B with a 4-bit base even at 2.2 GB, at the price of three times its 16-bit latency. These conclusions hold across the nine scenarios of the benchmark: neither quantisation, pruning nor the smaller model degrades some scenarios significantly more than others (permutation tests, $p \geq 0.15$), although the per-scenario samples are too small to exclude moderate differences (Appendix G).

What is specific to proactive agents. Three of our findings depend on the proactive setting and would not appear on a reactive benchmark. First, *silent passivity*: a configuration that never acts still scores an Acc-P of 0.470 and an F1-score of 0.468, so a score near this floor signals a non-functional agent rather than a weak one; checking MD and the parse rate of the proactive score (stage (a) of the error cascade) separates the two. Second, the failures of compressed models are *format* failures before they are capability failures: re-masked 50% magnitude pruning and Wanda 2:4 lose the proactive-score field in 92.5% and 34% of the

outputs, and mask-aware recovery removes these failures almost completely (Section 6.3). Third, most of the inference cost is spent on samples where the right decision is to do nothing: the fine-tuned models predict that no assistance is needed for 174–195 of the 400 test samples, and a model trained to emit the proactive score first keeps its proactive accuracy and could stop after 22 tokens on these samples, saving about a fifth of the latency (Section 6.2.6). For proactive agents, efficiency should therefore be evaluated together with the decision to act and the validity of the structured output, not only through an aggregate score.

7.3 Recommendations

For users and authors of proactive-agent benchmarks such as *ContextAgentBench*:

1. Re-train fine-tuned baselines under a documented configuration instead of comparing against reported scores, and tune the learning rate on a held-out part of the training split; it changes the F1-score by up to 0.09 here.
2. Report the effective batch size, the number of optimiser steps, the hardware parallelism and the seeds, and evaluate central configurations with several seeds and paired confidence intervals. With 400 test samples, differences below about 0.03 in F1-score cannot be resolved by a single run.
3. Report the share of unparseable outputs and the miss rate (MD) next to aggregate scores. On *ContextAgentBench*, an agent that never acts scores 0.470 Acc-P and 0.468 F1; scores near this floor indicate a non-functional agent, not a weak one.
4. Release every split used in the paper, state metric definitions as implemented, and score argument values, not only whether a tool call executes.

For compressing proactive reasoners:

5. To reduce memory, start with a smaller base model or a 4-bit base model, and account for the dequantisation latency of the runtime.
6. Keep the pruning mask during recovery: train the recovery update only on the unpruned weights rather than merging an unconstrained adapter and re-applying the mask.
7. Measure the cost of pruned models with the kernels used in deployment; dense storage gives no saving, and 2:4 kernels were slower at batch size one in our setting.

7.4 Limitations

Statistical power and single runs. All results come from one benchmark with 400 test samples, so a single evaluation has a 95% confidence interval of about ± 0.035 in F1-score. We trained three seeds for every configuration that supports a claim (Section 5.2), but the remaining sweeps (rank, target modules, further learning rates, quantisation at the released configuration and the pruning settings at $r = 8$) are single runs, and differences of a few hundredths between them should not be over-interpreted. The configuration sweeps were evaluated on the test split; the validation-based check in Section 6.2.1 suggests that this did not inflate our results.

Scope. We study one framework and one benchmark, and only the context-aware reasoner: the perception stage (vision-language model and speech recognition) is outside the cost measurements. Two claims of the original paper are outside this study (Table 8): the missing-sensor ablation cannot be reproduced from the release, which contains no code for it and only 32 *ContextAgentBench-Lite* samples with audio input, and the comparison with proprietary models requires paid access to models whose versions change over time.

Scope of cost measurements. Costs are measured for the reasoner only, with the bitsandbytes runtime on a single GPU at batch size one. Deployment runtimes and on-device measurements are out of scope.

Pruning. Pruning was evaluated on Llama-3.1-8B-Instruct only, structured pruning was restricted to MLP dimensions because preliminary experiments with attention-head pruning degraded the model severely, and recovery used LoRA rather than full fine-tuning. Mask-aware recovery triples the recovery training time in our implementation, because the masked update is materialised as a full matrix in every forward pass.

Knowledge distillation. We tested sequence-level distillation with and without filtering of the teacher outputs. A first attempt with a token-level objective had to truncate sequences to 1024 tokens to fit teacher and student into 24 GB, and none of its students learned the output format; distillation at the full length works, but adds nothing significant over direct fine-tuning (Table 6).

7.5 Future Work

The decision-first output order (Section 6.2.6) suggests a cheaper architecture for proactive agents: a small model that emits only the proactive score can gate the full reasoner, which is invoked only when assistance is needed; the 1–1.5B models already reach high proactive accuracy (Acc-P 0.903 ± 0.019 for Qwen2.5-1.5B) at a quarter of the memory. Mask-aware recovery combined with sparse kernels that are efficient at batch size one, or with batched monitoring of many users, could turn the preserved accuracy of pruned models into actual savings. Measuring cost per hour of monitoring rather than per sample, including the perception stage, would make the efficiency conclusions directly applicable to deployment.

Limitations "Test-set selection" and "Pruning": pruning rows over three seeds are running (32 runs); remove "each setting is evaluated from a single training run" for pruning once Tables 12/13/18 show mean ± std.

Pruning (Discussion and Limitations), Fig. 2, Introduction: structured pruning is being re-run at 5×10^{-5} (seeds 42/1/2). Check whether structured magnitude pruning still yields no parseable score (silent passivity example) and replace the 2×10^{-5} numbers.

Scope of cost measurements and Future Work: add GPTQ W4A16 on vLLM vs 16-bit vLLM, batched throughput (GPU-hours per 1,000 monitored user-hours) and the measured early exit (replaces the 22-token estimate) when jobs 913057, 913058 and 913085 finish.

Compute budget (Implementation Details): update the 550/900 GPU-hours once the remaining reviewer runs are finished and their costs are logged (sum of `costs.csv` plus `sacct` totals; the 900 covers scheduler accounting from 16 Sep 2026 under one account).

8 Conclusion

We reproduced *ContextAgent* on *ContextAgentBench* with the authors' released artefacts. The central claims hold: supervised fine-tuning with personas and thought traces clearly outperforms in-context learning, and the in-context baselines reproduce closely. The absolute fine-tuned scores, however, are not a stable reference. They depend on a learning rate that differs between the paper and the released code: with the paper's value, all three models score substantially higher than reported, and with the released value, two of the three do. Training with the effective batch size of eight-GPU data-parallel training reproduces the reported score of only one model. Users of the benchmark should therefore re-train the fine-tuned baselines under a documented configuration instead of comparing against the reported scores, tune the learning rate on held-out data, and report the effective batch size, the number of optimiser steps and the seeds.

Under resource constraints, the learning rate matters more than the LoRA configuration, and 4-bit quantisation of the base model halves inference memory without a detectable loss in task performance, at the cost of higher latency with on-the-fly dequantisation. A 1–1.5B base model fine-tuned at the full sequence length is the most effective way to reduce memory and latency, but it remains 0.03–0.10 F1 below a 7–8B model trained in the same way, and distillation does not close this gap. Pruning preserves performance up to 30% sparsity only if the sparsity survives the recovery step: restricting the recovery update to the unpruned weights reduces the loss at 50% and 2:4 sparsity from 0.10–0.27 to 0.03–0.09 F1, and with the paper's learning rate brings 50% unstructured sparsity to the level of the dense model, but without sparse kernels pruning brings no efficiency gain, and structured pruning yields efficiency gains only at a large and sometimes silent loss of proactive behaviour.

For proactive agents, the decision to act and the validity of the structured output are the first casualties of compression, and aggregate scores can hide them: an agent that never acts still answers every sample

that needs no assistance correctly. Efficient proactive agents therefore need to be evaluated, and designed, around the decision to act; emitting that decision first preserves it and saves about a fifth of the latency.

Broader Impact Statement

Proactive agents act on continuously sensed personal context, such as camera and audio input, phone data and persona information, which raises privacy concerns beyond those of reactive assistants. Our study uses only the publicly released *ContextAgentBench* and collects no new data. Its efficiency results cut both ways: smaller and quantised reasoners make it more feasible to process this context locally on the user's device rather than in the cloud, but they also lower the cost of always-on monitoring. Our failure analysis further shows that compressed agents can fail silently, by never acting or by producing unparseable decisions while keeping plausible aggregate scores; deployments should therefore check the decision to act and the validity of the structured output, not only aggregate task metrics.

Code availability

Code, configuration files and raw evaluation outputs are available at an anonymised repository for review: `https://anonymous.4open.science/r/tmlr-sub-eval/`.

References

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, 2024.

Taylor Berg-Kirkpatrick, David Burkett, and Dan Klein. An empirical investigation of statistical significance in NLP. In *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, pp. 995–1005, 2012.

Dan Biderman, Jacob Portes, Jose Javier Gonzalez Ortiz, Mansheej Paul, Philip Greengard, Connor Jennings, Daniel King, Sam Havens, Vitaliy Chiley, Jonathan Frankle, Cody Blakeney, and John P. Cunningham. LoRA learns less and forgets less. *Transactions on Machine Learning Research*, 2024.

Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, et al. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems*, volume 3, 2021.

Hongrong Cheng, Miao Zhang, and Javen Qinfeng Shi. A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):10558–10578, 2024. doi: 10.1109/TPAMI.2024.3447085.

Pierre V. Dantas, Lucas C. Cordeiro, and Waldir S. S. Junior. A review of state-of-the-art techniques for large language model compression. *Complex & Intelligent Systems*, 11(9):407, 2025. doi: 10.1007/s40747-025-02019-z.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, volume 35, pp. 30318–30332, 2022.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, pp. 10088–10115, 2023.

Christopher Diebel, Marc Goutier, Martin Adam, and Alexander Benlian. When AI-based agents are proactive: Implications for competence and system satisfaction in human–AI collaboration. *Business & Information Systems Engineering*, 68(2):259–278, 2025. doi: 10.1007/s12599-024-00918-y.

Jesse Dodge, Suchin Gururangan, Dallas Card, Roy Schwartz, and Noah A. Smith. Show your work: Improved reporting of experimental results. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 2185–2194, 2019. doi: 10.18653/v1/d19-1224.

Gongfan Fang, Xinyin Ma, Mingli Song, Michael Bi Mi, and Xinchao Wang. DepGraph: Towards any structural pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16091–16101, 2023.

Elias Frantar and Dan Alistarh. SparseGPT: Massive language models can be accurately pruned in one-shot. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 herd of models. arXiv preprint arXiv:2407.21783, 2024.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In *International Conference on Learning Representations*, 2024.

Odd Erik Gundersen and Sigbjørn Kjensmo. State of the art: Reproducibility in artificial intelligence. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.11503.

Daya Guo, Dejian Yang, Haowei Zhang, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645:633–638, 2025. doi: 10.1038/s41586-025-09422-z.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531, 2015.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6 (2):65–70, 1979.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Gun Il Kim, Sunga Hwang, and Beakcheol Jang. Efficient compressing and tuning methods for large language models: A systematic literature review. *ACM Computing Surveys*, 57(10):1–39, 2025. doi: 10.1145/3728636.

Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pp. 1317–1327, 2016. doi: 10.18653/v1/d16-1139.

Philipp Koehn. Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pp. 388–395, 2004.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023. doi: 10.1145/3600006.3613165.

Xudong Lu, Aojun Zhou, Yuhui Xu, Renrui Zhang, Peng Gao, and Hongsheng Li. SPP: Sparsity-preserved parameter-efficient fine-tuning for large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 33254–33269, 2024.

Yaxi Lu, Shenzhi Yang, Cheng Qian, Guirong Chen, Qinyu Luo, Yesai Wu, Huadong Wang, Xin Cong, Zhong Zhang, Yankai Lin, Weiwen Liu, Yasheng Wang, Zhiyuan Liu, Fangming Liu, and Maosong Sun. Proactive agent: Shifting LLM agents from reactive responses to active assistance. In *International Conference on Learning Representations*, 2025.

Xinyin Ma, Gongfan Fang, and Xinchao Wang. LLM-Pruner: On the structural pruning of large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Asit Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. Accelerating sparse deep neural networks. arXiv preprint arXiv:2104.08378, 2021.

Juan Pablo Muñoz, Jinjie Yuan, and Nilesh Jain. SQFT: Low-cost model adaptation in low-precision sparse foundation models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 12817–12832, 2024. doi: 10.18653/v1/2024.findings-emnlp.749.

Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164): 1–20, 2021.

Qwen Team. Qwen2.5 technical report. arXiv preprint arXiv:2412.15115, 2024.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, pp. 68539–68551, 2023.

Donald J. Schuirmann. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15(6):657–680, 1987. doi: 10.1007/bf01068419.

Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A simple and effective pruning approach for large language models. In *International Conference on Learning Representations*, 2024.

Bufang Yang, Lilin Xu, Liekang Zeng, Yunqi Guo, Siyang Jiang, Wenrui Lu, Kaiwei Liu, Hancheng Xiang, Xiaofan Jiang, Guoliang Xing, and Zhenyu Yan. ProAgent: Harnessing on-demand sensory contexts for proactive LLM agent systems. arXiv preprint arXiv:2512.06721, 2025a.

Bufang Yang, Lilin Xu, Liekang Zeng, Kaiwei Liu, Siyang Jiang, Wenrui Lu, Hongkai Chen, Xiaofan Jiang, Guoliang Xing, and Zhenyu Yan. ContextAgent: Context-aware proactive LLM agents with open-world sensory perceptions. In *Advances in Neural Information Processing Systems*, volume 38, pp. 185637–185671, 2025b.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi Feng, and Yongqiang Ma. LlamaFactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pp. 400–410, 2024. doi: 10.18653/v1/2024.acl-demos.38.

A Acronyms

Acc-Args Argument Accuracy

Acc-P Proactive Prediction Accuracy

FD False Detection rate

KD Knowledge Distillation

LLM Large Language Model

LoRA Low-Rank Adaptation

MD Missed Detection rate

MLP Multilayer Perceptron

NF4 4-bit NormalFloat

QLoRA Quantised Low-Rank Adaptation

RMSE Root Mean Square Error

TOST two one-sided tests

WANDA Pruning by **W**eights **and** **A**ctivations

B Training Costs

Table 9: Training cost of the reproduced *ContextAgent* models (released configuration). Runs with the exact released configuration (no LoRA dropout, no gradient checkpointing).

		Costs		
Method	Model	Duration (s)↓	GPU Peak (GB)↓	GPU Avg (GB)↓
ContextAgent	Llama-3.1-8B-Ins	2691.0	21.02	17.79
	DeepSeek-R1-7B	3446.6	21.02	16.68
	Qwen2.5-7B-Ins	2862.2	21.35	17.08

Table 10: Training cost under base-model quantisation (LoRA $r = 8$, learning rate 5×10^{-5}). All rows, including 16-bit, use the settings of the LoRA experiments (LoRA dropout 0.05, gradient checkpointing; Section 5.3), so durations and memory are not directly comparable to Table 9.

		Costs		
Model	Q-Bit	Duration (s)↓	GPU Peak (GB)↓	GPU Avg (GB)↓
LLaMA-3.1-8B-Ins	16-bit	**2855.8**	21.10	18.94
	8-bit	3656.8	19.07	18.75
	4-bit	3248.1	**11.28**	**10.77**
DeepSeek-R1-7B	16-bit	**2687.8**	21.12	18.12
	8-bit	3398.6	19.95	**13.6**
	4-bit	3474.7	**18.91**	16.15
Qwen2.5-7B-Ins	16-bit	**2935.6**	21.37	17.31
	8-bit	3609.9	19.95	**13.9**
	4-bit	3211.3	**15.95**	15.18

Table 11: Training cost of the combined configurations.

Model	Config	Costs		
		Duration (s)↓	GPU Peak (GB)↓	GPU Avg (GB)↓
LLaMA-3.1-8B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	3006.5	21.63	18.67
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	3305.7	12.24	11.73
DeepSeek-R1-7B	$r = 64, lr = 1.0 \times 10^{-4}$, all, 16-bit	2725.0	21.22	20.57
	$r = 64, lr = 1.0 \times 10^{-4}$, all, 4-bit	3402.6	11.54	11.16
Qwen2.5-7B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	2985.1	21.91	18.23
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	2996.5	15.26	14.69

Table 12: Recovery training cost after pruning (`Llama-3.1-8B-Instruct`). ‡Mask-aware recovery; rows marked 10^{-4} with learning rate 10^{-4} and the stated rank.

Structure	Method	%	Costs		
			Duration (s)↓	GPU Peak (GB)↓	GPU Avg (GB)↓
Unstructured	Wanda	10	2787.0	19.54	18.95
		30	2770.5	19.54	18.90
		50	2765.2	19.54	18.87
		50‡	8489.4	19.76	19.49
		50‡, 10^{-4}, r=8	8379.0	19.76	19.46
		50‡, 10^{-4}, r=32	8338.9	20.36	20.04
	Magnitude	10	2782.0	19.54	18.79
		30	2751.5	19.54	18.93
		50	2710.2	19.54	18.97
		50‡	8498.9	19.76	19.49
Structured	Activation	10	2891.5	19.71	18.84
		30	1986.3	15.33	14.55
		50	1479.3	11.14	10.64
	Magnitude	10	2745.0	19.71	18.83
		30	1926.3	15.33	14.43
		50	1524.0	11.14	10.36

C Additional Results

This appendix contains the results that are summarised in Section 6: the output analysis of the batch-64 runs, the single-run LoRA and quantisation sweeps at the released configuration, and the inference costs of all configurations.

C.1 Output Analysis of the Batch-64 Runs

Table 13: Effect of effective batch size 64 (≈ 50 optimiser steps, matching eight-GPU data-parallel training) versus batch size 8 (≈ 375 steps) and the values reported by Yang et al. (2025b). Batch-64 rows are single runs; batch-8 rows are the mean over three seeds (Table 2). †No output contains the required proactive score; the evaluation then treats every sample as "no assistance needed", so these scores equal those of a model that never acts (cf. structured magnitude pruning, Table 4).

		Proactive Predictions				Tool Calling			
Model	**Setting**	**Acc-P↑**	**MD↓**	**FD↓**	**RMSE↓**	**Precision↑**	**Recall↑**	**F1-score↑**	**Acc-Args↑**
Llama-3.1-8B-Ins	Reported	0.874	0.030	0.095	1.408	0.660	0.627	0.626	0.448
	Batch 64, 5×10^{-5}	0.885	0.055	0.060	1.353	0.635	0.593	0.591	0.263
	Batch 64, 10^{-4}	0.850	0.087	0.062	1.400	0.648	0.647	0.632	0.409
	Batch 8, 10^{-4} (ours)	0.927	0.040	0.033	1.024	0.800	0.785	0.777	0.664
DeepSeek-R1-7B	Reported	0.888	0.027	0.085	1.319	0.676	0.648	0.647	0.468
	Batch 64, 5×10^{-5}†	0.470	0.530	0.000	3.407	0.468	0.468	0.468	0.000
	Batch 64, 10^{-4}	0.750	0.068	0.182	1.933	0.419	0.439	0.409	0.091
	Batch 8, 10^{-4} (ours)	0.888	0.048	0.063	1.238	0.698	0.699	0.685	0.502
Qwen2.5-7B-Ins	Reported	0.894	0.013	0.091	1.264	0.672	0.644	0.645	0.459
	Batch 64, 5×10^{-5}	0.655	0.285	0.060	3.013	0.535	0.494	0.501	0.222
	Batch 64, 10^{-4}	0.840	0.077	0.083	1.404	0.568	0.542	0.540	0.283
	Batch 8, 10^{-4} (ours)	0.925	0.044	0.031	1.060	0.765	0.747	0.740	0.598

An analysis of the raw outputs shows why DeepSeek fails with an effective batch size of 64. With about 50 optimiser steps at the released learning rate (5×10^{-5}), it never adopts the trained output format: none of its 400 outputs contains the required proactive-score and tool-calling fields; 72% answer in the free-form style of the base model and 28% never close the reasoning trace. At 10^{-4}, the format is largely learned (96% of outputs are parseable), but the decisions remain poor: false alarms (FD) rise from 6% to 18%, 10% of the outputs contain missing or malformed tool calls (batch size 8: 1%), and only 2% of all samples receive exactly the correct tool set (batch size 8: 15%). With 375 optimiser steps at batch size 8, these failures are rare. Llama, in contrast, learns the output format even at batch size 64 (at most 3% unparseable outputs); its lower scores come from less accurate tool selection (36–38% wrong or incomplete tool names). Qwen shows the same failure as DeepSeek in a milder form: at batch size 64 and 5×10^{-5}, 48% of its outputs contain no proactive score (batch size 8: 0%), which lowers its tool-calling F1 to 0.501. At 10^{-4} it learns the format (all outputs parseable), but its decisions remain worse than at batch size 8: false alarms rise from 3% to 8%, missed detections from 4% to 8%, and only 4% of the samples receive exactly the correct tool set (batch size 8: 14%). With about 50 optimiser steps, the released learning rate is therefore too low for two of the three models to learn the output format reliably, and even at 10^{-4} the decisions of DeepSeek and Qwen remain clearly worse than with 375 steps; DeepSeek, whose base model produces long free-form reasoning, is affected most.

C.2 Inference Cost of the Reproduced Models

Table 14: Inference cost of the reproduced ContextAgent models (16-bit, released configuration). Measured on the runs with the exact released configuration (Section 5.3).

		Costs					
Method	Model	GPU Peak (GB) ↓	GPU Avg (GB) ↓	Avg Batch (s) ↓	Min Batch (s) ↓	Max Batch (s) ↓	Avg Samples/s ↑
ContextAgent	Llama-3.1-8B-Ins	15.6	15.6	9.0	4.0	21.7	0.112
	DeepSeek-R1-7B	14.8	14.8	8.7	2.4	23.1	0.115
	Qwen2.5-7B-Ins	14.8	14.8	8.3	1.7	22.6	0.121

The three models need 14.8–15.6 GB at inference and process one sample in 8.3–9.0 s on average.

C.3 LoRA Rank, Target Modules and Learning Rate

Table 15: Effect of the LoRA rank r (learning rate 5×10^{-5}, all linear layers; $r = 8$ is the released default). All rows are single runs (seed 42); the $r = 8$ row is the seed-42 run of Table 1.

		Proactive Predictions				Tool Calling			
Model	Rank	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	8	0.917	0.045	0.037	1.129	0.762	0.739	0.735	0.595
	16	**0.938**	**0.030**	**0.033**	1.033	**0.803**	**0.795**	0.782	0.658
	32	0.935	0.033	**0.033**	1.004	0.801	0.784	0.777	0.683
	64	0.935	0.033	**0.033**	**0.937**	0.802	0.793	**0.783**	**0.685**
DeepSeek-R1-7B	8	0.873	0.048	0.080	1.365	0.625	0.622	0.609	0.368
	16	0.880	0.060	0.060	1.295	0.657	0.661	0.646	0.433
	32	0.897	0.045	**0.058**	1.278	0.694	0.697	0.678	**0.490**
	64	**0.907**	**0.033**	0.060	**1.160**	**0.701**	**0.710**	**0.690**	0.489
Qwen2.5-7B-Ins	8	0.915	0.035	0.050	1.137	0.734	0.719	0.708	0.551
	16	0.922	0.037	0.040	1.075	0.774	0.742	0.739	0.618
	32	**0.943**	0.033	**0.025**	**0.927**	**0.798**	**0.782**	**0.770**	0.631
	64	0.935	**0.030**	0.035	0.987	0.795	0.771	0.769	**0.649**

Table 15 presents the effect of LoRA rank on task performance across all three models, with all other hyperparameters fixed to their baseline values. For `Llama-3.1-8B-Instruct`, the F1-score rises from 0.735 at $r = 8$ to 0.782 at $r = 16$ and then plateaus (0.777 at $r = 32$, 0.783 at $r = 64$). `Qwen2.5-7B-Instruct` improves up to $r = 32$, with F1-scores of 0.708, 0.739, 0.770 and 0.769. *DeepSeek-R1-7B* improves at every rank, from 0.609 at $r = 8$ to 0.690 at $r = 64$, although the step from $r = 32$ (0.678) to $r = 64$ is within run-to-run uncertainty.

Table 16: LoRA target modules ($r = 8$, learning rate 5×10^{-5}).

		Proactive Predictions				Tool Calling			
Model	Target modules	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	All linear	**0.917**	0.045	**0.037**	**1.129**	**0.762**	**0.739**	**0.735**	**0.595**
	Attention only	0.890	0.065	0.045	1.262	0.723	0.707	0.699	0.512
	MLP only	0.905	**0.037**	0.058	1.224	0.731	0.714	0.706	0.556
DeepSeek-R1-7B	All linear	**0.873**	**0.048**	0.080	**1.365**	**0.625**	**0.622**	**0.609**	**0.368**
	Attention only	0.823	0.058	0.120	1.527	0.516	0.519	0.506	0.231
	MLP only	**0.873**	0.050	**0.077**	1.384	0.613	0.599	0.594	0.361
Qwen2.5-7B-Ins	All linear	**0.915**	**0.035**	0.050	1.137	**0.734**	**0.719**	0.708	0.551
	Attention only	0.877	0.062	0.060	1.312	0.643	0.628	0.620	0.412
	MLP only	**0.915**	0.048	**0.037**	**1.124**	0.732	0.715	**0.709**	**0.562**

Table 16 reports the effect of restricting LoRA adapters to specific architectural components, evaluated at the baseline rank and learning rate. For `Llama-3.1-8B-Instruct`, adapting all linear layers gives the best results (F1-score 0.735), and adapting only the attention projections gives the worst (0.699). For `Qwen2.5-7B-Instruct`, adapting only the MLP layers is as good as adapting all linear layers (0.709 versus 0.708), while attention-only adaptation is clearly worse (0.620). *DeepSeek-R1-7B* shows the same pattern: adapting only the MLP layers comes close to adapting all linear layers (F1-score 0.594 versus 0.609), whereas attention-only adaptation loses 0.10 (0.506) and raises false detections to 0.120.

Table 17: Learning rate (LoRA $r = 8$, all linear layers). Scores recomputed from the released prediction files. The 5×10^{-5} and 10^{-4} rows show the mean ± standard deviation over three training seeds (42, 1, 2); the 10^{-5} and 2×10^{-4} rows are single runs, except Qwen at 2×10^{-4}, which is the mean over seed 42 and the two validation-selected runs with the same configuration (seeds 1 and 2; Section 6.2.1).

		Proactive Predictions				Tool Calling			
Model	LR	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	1.0×10^{-5}	0.885	**0.035**	0.080	1.253	0.668	0.638	0.634	0.415
	5.0×10^{-5} (code)	0.915±0.007	0.044±0.011	0.041±0.006	1.124±0.033	0.764±0.006	0.745±0.006	0.740±0.005	0.593±0.008
	1.0×10^{-4} (paper)	0.927±0.016	0.040±0.007	0.033±0.009	1.024±0.077	0.800±0.008	0.785±0.006	0.777±0.005	0.664±0.004
	2.0×10^{-4}	**0.930**	0.040	**0.030**	**0.999**	**0.805**	**0.796**	**0.786**	**0.685**
DeepSeek-R1-7B	1.0×10^{-5}	0.868	0.058	0.075	1.367	0.621	0.598	0.598	0.376
	5.0×10^{-5} (code)	0.855±0.019	0.056±0.008	0.089±0.016	1.431±0.084	0.613±0.013	0.608±0.019	0.596±0.015	0.365±0.008
	1.0×10^{-4} (paper)	**0.888±0.010**	**0.048±0.009**	**0.063±0.011**	**1.238±0.031**	0.698±0.008	0.699±0.011	0.685±0.006	0.502±0.003
	2.0×10^{-4}	0.882	0.052	0.065	1.273	**0.720**	**0.721**	**0.704**	**0.527**
Qwen2.5-7B-Ins	1.0×10^{-5}	0.825	0.062	0.113	1.564	0.580	0.525	0.535	0.337
	5.0×10^{-5} (code)	0.912±0.010	0.041±0.008	0.048±0.004	1.129±0.052	0.732±0.006	0.716±0.010	0.707±0.008	0.546±0.016
	1.0×10^{-4} (paper)	0.925±0.008	0.044±0.009	**0.031±0.004**	1.060±0.017	0.765±0.006	0.747±0.002	0.740±0.005	0.598±0.024
	2.0×10^{-4}	**0.927±0.007**	**0.037±0.008**	0.036±0.004	**1.026±0.062**	**0.796±0.008**	**0.784±0.006**	**0.774±0.008**	**0.645±0.016**

Table 17 and Figure 1 compare four learning rates: 10^{-5}, the released 5×10^{-5}, the paper's 10^{-4} and 2×10^{-4}. Above 5×10^{-5}, tool-calling performance increases with the learning rate for all three models. Moving from the released 5×10^{-5} to the paper's 10^{-4} raises the F1-score from 0.740 to 0.777 (Llama), 0.596 to 0.685 (DeepSeek) and 0.707 to 0.740 (Qwen), averaged over three seeds; all three differences are significant (paired bootstrap: +0.037, +0.089 and +0.033, with 95% confidence intervals excluding zero), and the seed standard deviation of the F1-score is at most 0.015. At 2×10^{-4}, all three models improve further (Llama 0.786, DeepSeek 0.704, Qwen 0.774 ± 0.008), although the gain over 10^{-4} is small for Llama (0.009). 10^{-5} is clearly worse for Llama and Qwen (0.634 and 0.535), whereas for DeepSeek it is on par with the released value (0.598 versus 0.596). The released value is therefore not anomalous: it lies on the lower side of an increasing trend. This is consistent with the models being undertrained at the released setting, given only 600 training samples (Section 5.3). Since 2×10^{-4} is the best tested value for all three models, we cannot exclude that larger learning rates perform better still.

C.4 Quantisation at the Released Configuration

Table 18: Task performance under base-model quantisation (LoRA $r = 8$, learning rate 5×10^{-5}, all linear layers). All rows are single runs (seed 42); the 16-bit row is the seed-42 run of Table 1.

		Proactive Predictions				Tool Calling			
Model	Q-Bit	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Llama-3.1-8B-Ins	16-bit	0.917	0.045	0.037	1.129	0.762	0.739	0.735	0.595
	8-bit	0.912	0.055	0.033	1.150	0.775	0.741	0.742	0.608
	4-bit	0.892	0.037	0.070	1.256	0.736	0.712	0.708	0.577
DeepSeek-R1-7B	16-bit	0.873	0.048	0.080	1.365	0.625	0.622	0.609	0.368
	8-bit	0.865	0.065	0.070	1.414	0.631	0.631	0.615	0.373
	4-bit	0.843	0.060	0.098	1.476	0.619	0.600	0.596	0.385
Qwen2.5-7B-Ins	16-bit	0.915	0.035	0.050	1.137	0.734	0.719	0.708	0.551
	8-bit	0.915	0.050	0.035	1.161	0.753	0.722	0.720	0.542
	4-bit	0.863	0.090	0.048	1.620	0.696	0.693	0.680	0.498

Table 18 compares 16-bit LoRA, 8-bit LLM.int8 and 4-bit NF4 QLoRA, with all other settings at the released default. 8-bit quantisation does not measurably reduce task performance: the F1-score changes by +0.007 (Llama), +0.006 (DeepSeek) and +0.012 (Qwen) relative to 16-bit, and none of these differences are significant (all 95% confidence intervals include zero). 4-bit NF4 lowers the F1-score by 0.013–0.028 (Llama 0.735 to 0.708, DeepSeek 0.609 to 0.596, Qwen 0.708 to 0.680); these differences are also not significant individually. The only nominally significant effect, a lower Acc-P for 4-bit Qwen (−0.052, 95% CI [−0.090, −0.015]), does

not survive the Holm correction over the quantisation comparisons. With single runs, the evaluation is not precise enough to establish equivalence within ±0.03 (TOST), so we report these results as no detectable difference rather than as equivalence. These results agree with what is known for QLoRA on other tasks (Dettmers et al., 2023): base-model quantisation retains most of the fine-tuned performance, and we include them as a supporting cost–performance reference for the reproduced models rather than as a new finding.

The training resource requirements corresponding to these configurations are reported in Table 10 in Appendix B. For training, 4-bit quantisation reduces peak GPU memory by 47% for Llama (21.10 to 11.28 GB), but only by 25% for Qwen (21.37 to 15.95 GB) and 10% for DeepSeek (21.12 to 18.91 GB). Quantised training is also slower, by 9–29% at 4-bit and 23–28% at 8-bit.

Table 19: Inference cost under base-model quantisation (configuration as in Table 18). The 16-bit rows repeat Table 14.

		Costs					
Model	Q-Bit	GPU Peak (GB) ↓	GPU Avg (GB) ↓	Avg Batch (s) ↓	Min Batch (s) ↓	Max Batch (s) ↓	Avg Samples/s ↑
LLaMA-3.1-8B-Ins	16-bit	15.6	15.6	9.0	4.0	21.7	0.112
	8-bit	9.3	9.3	17.4	7.1	45.8	0.057
	4-bit	8.2	8.1	11.4	4.9	25.2	0.088
DeepSeek-R1-7B	16-bit	14.8	14.8	8.7	2.4	23.1	0.115
	8-bit	11.0	9.9	18.2	6.8	55.0	0.055
	4-bit	7.7	7.7	10.2	4.4	25.3	0.098
Qwen2.5-7B-Ins	16-bit	14.8	14.8	8.3	1.7	22.6	0.121
	8-bit	9.6	9.2	18.5	3.5	45.6	0.054
	4-bit	7.7	7.7	11.8	0.7	81.2	0.085

At inference (Table 19), peak GPU memory decreases with the bit width for all three models; for Llama it falls from 15.6 GB at 16-bit to 9.3 GB at 8-bit and 8.2 GB at 4-bit (40% and 47% reductions). Latency does not follow the same ordering. With the bitsandbytes kernels used here, LLM.int8 roughly doubles the average time per batch compared to 16-bit, while 4-bit NF4 is 17–42% slower than 16-bit but clearly faster than 8-bit.

C.5 Costs of the Combined Configurations

Training costs for these configurations are in Table 11 (Appendix B). At 4-bit, peak training memory falls from 21.63 to 12.24 GB for Llama, from 21.22 to 11.54 GB for DeepSeek and from 21.91 to 15.26 GB for Qwen, with training time increasing by 10% for Llama, 25% for DeepSeek and less than 1% for Qwen.

Table 20: Inference cost of the combined configurations.

		Costs					
Model	Config	GPU Peak (GB) ↓	GPU Avg (GB) ↓	Avg Batch (s) ↓	Min Batch (s) ↓	Max Batch (s) ↓	Avg Samples/s ↑
Llama-3.1-8B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	15.6	15.6	**9.1**	**4.2**	**21.7**	**0.109**
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	**8.3**	**8.3**	13.9	5.5	33.9	0.072
DeepSeek-R1-7B	$r = 64, lr = 1.0 \times 10^{-4}$, all, 16-bit	14.8	14.8	8.5	3.5	20.3	0.117
	$r = 64, lr = 1.0 \times 10^{-4}$, all, 4-bit	7.8	7.8	8.6	0.5	18.8	0.116
Qwen2.5-7B-Ins	$r = 32, lr = 1.0 \times 10^{-4}$, all, 16-bit	14.8	14.8	**8.5**	**3.5**	**19.5**	**0.118**
	$r = 32, lr = 1.0 \times 10^{-4}$, all, 4-bit	**7.7**	**7.7**	11.3	4.4	29.2	0.089

At inference (Table 20), 4-bit quantisation reduces peak GPU memory by 47–48% for all three models (Llama 15.6 to 8.3 GB, DeepSeek 14.8 to 7.8 GB, Qwen 14.8 to 7.7 GB). For Llama and Qwen this comes with a

53% and 33% increase in average batch time; for DeepSeek the average batch time is almost unchanged (8.5 versus 8.6 s).

C.6 Inference Cost after Pruning

Table 21: Inference cost after pruning and recovery. ‡Mask-aware recovery; rows marked 10^{-4} with learning rate 10^{-4} and the stated rank.

			Costs					
Structure	Method	%	GPU Peak (GB) ↓	GPU Avg (GB) ↓	Avg Batch (s) ↓	Min Batch (s) ↓	Max Batch (s) ↓	Avg Samples/s ↑
Unstructured	Wanda	10	15.6	15.6	8.7	3.8	19.3	0.1154
		30	15.6	15.6	8.1	3.9	20.1	0.1233
		50	15.7	15.7	8.2	3.9	68.0	0.1227
		50‡	15.6	15.6	9.6	4.1	31.5	0.1043
		50‡, 10^{-4}, r=8	15.6	15.6	9.2	2.9	22.2	0.1086
		50‡, 10^{-4}, r=32	15.6	15.6	9.8	4.5	23.4	0.1018
	Magnitude	10	15.6	15.6	8.7	3.8	23.5	0.1148
		30	15.6	15.6	8.1	3.9	17.1	0.1227
		50	15.7	15.7	10.2	2.9	68.9	0.0981
		50‡	15.7	15.6	10.0	4.3	61.0	0.1000
Structured	Activation	10	13.2	13.2	8.2	3.7	17.8	0.1220
		30	8.9	8.9	5.4	2.3	13.3	0.1850
		50	5.7	5.6	5.3	1.5	32.1	0.1883
	Magnitude	10	13.3	13.3	34.8	4.6	62.0	0.0287
		30	9.0	9.0	43.7	42.6	47.6	0.0229
		50	5.7	5.6	31.4	28.2	41.4	0.0318

At inference (Table 21), unstructured pruning leaves memory and latency unchanged because the pruned weights are stored and multiplied densely; this also holds for mask-aware recovery at 10^{-4} (15.6 GB, 9.2–9.8 s per sample). Structured magnitude pruning is 3.5–5 times *slower* than the dense model despite its smaller size. This slowdown coincides with degenerate generation: for the 10%, 30% and 50% settings, none of the 400 outputs contains a proactive score; the outputs consist of repeated fragments of quotes, braces and tool-schema words. Re-tokenising the saved outputs gives on average 463, 860 and 969 tokens per sample, against 132 for the dense model; for degenerate text this is a lower bound, because runs of whitespace and punctuation merge into fewer tokens than were generated. Calibrating against the per-sample latencies of the activation-pruned model of the same size, whose latency grows linearly with output length ($r \geq 0.999$), shows that all 400 samples at 30% and most samples at 50% (implied mean length 1,024 tokens) run until the limit of 1,024 new tokens, as do about one third (130) at 10%. The slowdown therefore comes from generating up to eight times as many tokens per sample, not from slower computation per token. The dense model and the activation-pruned models stay far below the limit (on average 127–168 tokens; only 15 of the 400 samples at 50% activation pruning reach it). Hardware support for semi-structured sparsity does not change this picture in our setting: executing the 2:4 models with PyTorch's sparse tensor-core kernels leaves the task metrics unchanged, also after mask-aware recovery, but makes generation about 2.8 times slower at batch size one (24.4–25.3 s per sample), and the measured peak memory stays at 15.8 GB because the weights are converted only after loading (Table 24, Appendix D).

D Semi-structured Pruning

Semi-structured pruning aims to balance the flexibility of unstructured pruning with the hardware efficiency of structured sparsity. A widely adopted formulation is **N:M** sparsity, in which exactly N non-zero weights are retained within each contiguous block of M parameters (Mishra et al., 2021). This specific pattern is highly relevant for modern hardware accelerators, such as GPUs equipped with Sparse Tensor Cores, which natively accelerate 2:4 sparsity to achieve tangible inference speedups.

Table 22: Semi-structured pruning (50% sparsity) of `Llama-3.1-8B-Instruct` after recovery fine-tuning. The pruning mask is re-applied after merging the recovery adapter; re-masked Wanda rows show the mean of three evaluations. ‡Mask-aware recovery: the LoRA update is restricted to the unpruned weights (Section 5.5). Rows marked 10^{-4}: mask-aware recovery with learning rate 10^{-4} and the stated rank instead of the released recovery configuration ($r = 8$, 5×10^{-5}); the two rows with $r = 32$ are the mean ± standard deviation over three seeds, all other mask-aware rows are single runs.

			Proactive Predictions				Tool Calling			
Structure	Pruning	%	Acc-P↑	MD↓	FD↓	RMSE↓	Precision↑	Recall↑	F1-score↑	Acc-Args↑
Semi-structured	Wanda	2:4 (50%)	0.703	0.162	0.135	2.291	0.511	0.458	0.469	0.185
		4:8 (50%)	0.862	0.094	0.044	1.423	0.669	0.608	0.620	0.432
		2:4 (50%)‡	0.875	0.045	0.080	1.343	0.664	0.665	0.649	0.455
		4:8 (50%)‡	0.868	0.083	0.050	1.323	0.688	0.673	0.666	0.490
		2:4 (50%)‡, 10^{-4}, r=8	0.900	0.043	0.058	1.216	0.725	0.730	0.711	0.552
		4:8 (50%)‡, 10^{-4}, r=8	0.927	0.043	**0.030**	1.034	0.763	0.756	0.745	0.587
		2:4 (50%)‡, 10^{-4}, r=32	0.901±0.014	0.053±0.008	0.046±0.007	1.204±0.098	0.736±0.019	0.729±0.026	0.719±0.022	0.567±0.021
		4:8 (50%)‡, 10^{-4}, r=32	**0.929±0.009**	**0.041±0.009**	**0.030±0.000**	**1.025±0.084**	**0.775±0.007**	**0.773±0.015**	**0.756±0.008**	**0.613±0.005**
	Magnitude	2:4 (50%)	0.805	0.080	0.115	1.740	0.530	0.518	0.517	0.276
		4:8 (50%)	0.767	0.163	0.070	1.810	0.571	0.538	0.544	0.305
		2:4 (50%)‡	0.875	0.072	0.052	1.388	0.680	0.676	0.666	0.473
		4:8 (50%)‡	0.882	0.062	0.055	1.339	0.711	0.707	0.693	0.522

Table 22 shows semi-structured pruning at 50% sparsity. With the original recovery and the pruning mask re-applied afterwards, all four settings lose substantially against the dense model (seed-42 run, F1-score 0.735; all differences significant). The less constrained 4:8 pattern is better than 2:4 for WANDA (F1-score 0.620 versus 0.469, Acc-Args 0.432 versus 0.185) and slightly better for magnitude pruning (0.544 versus 0.517); with 2:4, both criteria fail similarly, with Acc-P falling to 0.70–0.81. Mask-aware recovery (rows marked ‡) improves all four settings significantly, by +0.046 (WANDA 4:8) to +0.180 (WANDA 2:4) in F1-score, and removes the collapse of the 2:4 pattern: all four settings then reach an F1-score of 0.649–0.693 and an Acc-P of at least 0.868. The differences between the two criteria and the two patterns shrink to at most 0.044 in F1-score, and magnitude 4:8 performs best (0.693). All four remain significantly below the dense model (−0.042 to −0.086) and slightly below unstructured WANDA at the same sparsity with mask-aware recovery (0.706).

With the paper's learning rate of 10^{-4}, mask-aware recovery of the WANDA patterns improves further. At $r = 8$ the F1-score rises to 0.711 (2:4) and 0.745 (4:8), significantly above mask-aware recovery at 5×10^{-5} (+0.062, 95% CI $[0.024, 0.100]$, and +0.079, $[0.045, 0.114]$); at $r = 32$ it reaches 0.719 ± 0.022 (2:4) and 0.756 ± 0.008 (4:8; mean ± standard deviation over three seeds). Against the dense model trained at the same learning rate ($r = 8$, three seeds, F1-score 0.777), 4:8 with $r = 32$ shows no significant difference in F1-score (-0.021, $[-0.043, 0.001]$) or Acc-P, but a lower Acc-Args (-0.051, $[-0.084, -0.018]$). 2:4 remains significantly below the dense model (-0.058, $[-0.081, -0.034]$); its seed-42 run (0.744) was the best of the three seeds, so a single run would have overstated it. 2:4 is also the only mask-aware setting with a noticeable share of outputs without a parseable proactive score (1.7% on average over the three seeds, Section F.1). The rows with $r = 8$ are single runs.

Table 23: Recovery training cost after semi-structured pruning. ‡Mask-aware recovery; rows marked 10^{-4} with learning rate 10^{-4} and the stated rank.

Structure	Method	%	Costs		
			Duration (s)↓	GPU Peak (GB)↓	GPU Avg (GB)↓
Semi-structured	Wanda	2:4 (50%)	2719.8	**19.54**	18.82
		4:8 (50%)	2732.5	**19.54**	18.91
		2:4 (50%)‡	8437.5	19.76	19.50
		4:8 (50%)‡	8468.4	19.76	19.48
		2:4 (50%)‡, 10^{-4}, r=8	8430.2	19.76	19.48
		4:8 (50%)‡, 10^{-4}, r=8	8298.0	19.76	19.48
		2:4 (50%)‡, 10^{-4}, r=32	8442.4	20.36	20.07
		4:8 (50%)‡, 10^{-4}, r=32	8320.7	20.36	20.14
	Magnitude	2:4 (50%)	**2700.6**	**19.54**	18.96
		4:8 (50%)	2726.3	**19.54**	18.89
		2:4 (50%)‡	8478.8	19.76	19.46
		4:8 (50%)‡	8479.0	19.76	19.48

Training costs are reported in Table 23. With the original recovery, peak GPU memory is 19.54 GB for all configurations, as for unstructured pruning, and training times range from 2701 to 2733 seconds. Mask-aware recovery needs about 3.1 times as long (8298–8479 s) at a slightly higher peak of 19.76 GB (20.36 GB with $r = 32$), because the masked update is formed as a full matrix in every forward pass; the learning rate does not change this cost.

Table 24: Inference cost after semi-structured pruning. "Dense storage": the pruned weights are stored and multiplied densely. "Sparse kernels": the 2:4 layers are executed with PyTorch's semi-structured sparse tensor-core kernels (`to_sparse_semi_structured`). ‡Mask-aware recovery; rows marked 10^{-4} with learning rate 10^{-4} and the stated rank.

Structure	Method	%	Costs					
			GPU Peak (GB)↓	GPU Avg (GB)↓	Avg Batch (s)↓	Min Batch (s)↓	Max Batch (s)↓	Avg Samples/s↑
Semi-structured (dense storage)	Wanda	2:4 (50%)	15.7	15.7	8.7	1.9	69.0	0.1150
		4:8 (50%)	15.7	15.7	7.6	3.0	68.4	0.1324
		2:4 (50%)‡	15.6	15.6	9.6	4.0	37.8	0.1040
		4:8 (50%)‡	15.6	15.6	10.3	4.4	27.5	0.0975
		2:4 (50%)‡, 10^{-4}, r=8	15.6	15.6	9.7	4.1	28.3	0.1027
		4:8 (50%)‡, 10^{-4}, r=8	15.6	15.6	9.8	3.1	26.3	0.1019
		2:4 (50%)‡, 10^{-4}, r=32	15.6	15.6	10.3	3.0	23.9	0.0971
		4:8 (50%)‡, 10^{-4}, r=32	15.6	15.6	9.9	4.0	23.1	0.1009
	Magnitude	2:4 (50%)	15.7	15.7	9.0	3.6	69.0	0.1116
		4:8 (50%)	15.7	15.7	8.2	2.3	68.6	0.1223
		2:4 (50%)‡	15.7	15.6	9.4	4.1	68.7	0.1061
		4:8 (50%)‡	15.6	15.6	9.3	4.0	28.5	0.1078
Sparse kernels	Wanda	2:4 (50%)	15.8	15.8	24.4	3.8	192.0	0.0410
		2:4 (50%)‡	15.8	15.8	24.9	9.6	57.9	0.0402
	Magnitude	2:4 (50%)	15.8	15.8	24.6	8.7	192.7	0.0407
		2:4 (50%)‡	15.8	15.8	25.3	10.1	54.7	0.0395

At inference (Table 24), peak GPU memory remains 15.6–15.7 GB and the average batch time 7.6–10.3 s, similar to the dense model (9.0 s), also with mask-aware recovery; differences come from output length, not from the kernels. To test whether hardware support changes this, we executed the 2:4 models with PyTorch's semi-structured sparse kernels, which use the sparse tensor cores of the L4 GPU: all 224 linear layers of each model are converted after loading, and the task metrics are unchanged within evaluation noise (re-masked: F1-score 0.487 and 0.503; mask-aware: 0.645 and 0.682, not significantly different from the same models with dense execution, -0.004, 95% CI $[-0.042, 0.033]$, and $+0.016$, $[-0.019, 0.052]$). Generation, however, becomes about 2.8 times slower (24.4–25.3 s per sample instead of 8.7–9.6 s), most likely because with a batch size of one every decoding step multiplies a single token vector, a shape for which the sparse kernels are less efficient than dense matrix multiplication. The compressed weights reduce the allocated GPU memory

from 15.0 to 9.3 GiB (−38%), but because the dense weights are loaded first and converted afterwards, the memory reserved by the process, and thus the measured peak, stays at 15.8 GB. Realising both savings would require loading the weights directly in compressed form and batched inference.

How much of the recovery update re-masking discards. For each unconstrained recovery adapter and its pruned base model, with mask M (1 = kept) and update $D = BA$, we measure the share of the update that re-applying the mask removes, $\|(1 - M) \odot D\|_F^2 / \|D\|_F^2$, summed over all 224 adapted layers (Table 25). For every adapter it equals the sparsity, and it is the same for all module types: the unconstrained update is spread evenly over pruned and kept weights, so re-masking discards a share equal to the sparsity. Mask-aware recovery avoids this loss by construction, which is why it helps most at 50% and with the 2:4 pattern.

Table 25: Share of the unconstrained recovery update removed by re-applying the pruning mask ($\|(1-M)\odot BA\|_F^2/\|BA\|_F^2$ over all adapted layers), and its range over the seven module types. "Measured sparsity": share of pruned weights in the adapted layers (magnitude pruning thresholds globally).

Criterion	Pattern	Measured sparsity	Discarded share	Range over modules
Wanda	10%	0.100	0.099	0.099–0.100
	30%	0.300	0.298	0.297–0.299
	50%	0.500	0.497	0.496–0.499
	2:4	0.500	0.498	0.497–0.499
	4:8	0.500	0.497	0.496–0.499
Magnitude	10%	0.100	0.100	0.100–0.102
	30%	0.301	0.301	0.300–0.306
	50%	0.502	0.502	0.501–0.512
	2:4	0.500	0.500	0.500–0.501
	4:8	0.500	0.500	0.500–0.501

E Statistical Analysis

Figure 3 shows the paired-bootstrap differences in tool-calling F1-score, with 95% confidence intervals, for all comparisons discussed in the paper (method in Section 5.6), and Table 26 lists every comparison with its confidence intervals for F1-score, Acc-P and Acc-Args.

Table 26: All paired comparisons reported in the paper: difference between the two configurations of each comparison (first minus second as named: "A vs B" is A minus B) with 95% paired-bootstrap confidence interval over the 400 test samples; *: the interval excludes zero. Holm: for effects significant before correction, whether the F1 effect survives the Holm–Bonferroni correction within its group. Generated by `scripts/comparison_table.py`.

Comparison	Δ F1-score [95% CI]	Δ Acc-P [95% CI]	Δ Acc-Args [95% CI]	Holm
Learning rate: paper (10^{-4}) vs released (5×10^{-5}), 3 vs 3 seeds				
Llama-8B	+0.037 [+0.016, +0.057]*	+0.012 [−0.003, +0.028]	+0.071 [+0.041, +0.102]*	yes
DeepSeek-7B	+0.089 [+0.065, +0.114]*	+0.033 [+0.010, +0.057]*	+0.137 [+0.106, +0.169]*	yes
Qwen-7B	+0.033 [+0.013, +0.054]*	+0.013 [−0.002, +0.028]	+0.052 [+0.021, +0.084]*	yes
Exact released configuration vs our 5×10^{-5} runs (2 vs 3 runs)				
Llama-8B	−0.021 [−0.042, +0.000]	−0.006 [−0.025, +0.012]	−0.012 [−0.043, +0.019]	
DeepSeek-7B	−0.004 [−0.030, +0.023]	−0.005 [−0.029, +0.019]	−0.004 [−0.035, +0.027]	
Qwen-7B	+0.000 [−0.021, +0.021]	+0.001 [−0.017, +0.018]	−0.008 [−0.038, +0.023]	
Validation-selected configuration (2×10^{-4}, 3 seeds) vs test-chosen and specified configurations				
Llama-8B vs combined	+0.025 [+0.007, +0.044]*	+0.009 [−0.005, +0.024]	+0.021 [−0.008, +0.050]	yes
Llama-8B vs paper config.	+0.018 [−0.001, +0.038]	+0.010 [−0.005, +0.026]	+0.014 [−0.018, +0.045]	
Llama-8B vs released config.	+0.055 [+0.034, +0.077]*	+0.022 [+0.008, +0.037]*	+0.085 [+0.054, +0.116]*	yes
DeepSeek-7B vs combined	+0.005 [−0.014, +0.023]	−0.004 [−0.022, +0.013]	+0.028 [+0.001, +0.054]*	
DeepSeek-7B vs paper config.	+0.017 [−0.007, +0.041]	−0.001 [−0.022, +0.020]	+0.041 [+0.009, +0.073]*	
DeepSeek-7B vs released config.	+0.106 [+0.081, +0.132]*	+0.033 [+0.010, +0.056]*	+0.178 [+0.145, +0.214]*	yes
Qwen-7B vs combined	+0.021 [+0.003, +0.040]*	+0.002 [−0.012, +0.017]	+0.020 [−0.007, +0.049]	no
Qwen-7B vs paper config.	+0.034 [+0.014, +0.055]*	+0.003 [−0.012, +0.018]	+0.046 [+0.016, +0.078]*	yes
Qwen-7B vs released config.	+0.067 [+0.045, +0.090]*	+0.016 [−0.002, +0.034]	+0.098 [+0.067, +0.131]*	yes

Comparison	Δ F1-score [95% CI]	Δ Acc-P [95% CI]	Δ Acc-Args [95% CI]	Holm
Base-model quantisation at the released configuration (single runs)				
Llama-8B, 8-bit vs 16-bit	+0.007 [−0.020, +0.033]	−0.005 [−0.032, +0.023]	+0.013 [−0.030, +0.055]	
DeepSeek-7B, 8-bit vs 16-bit	+0.006 [−0.031, +0.044]	−0.008 [−0.040, +0.027]	+0.005 [−0.043, +0.054]	
Qwen-7B, 8-bit vs 16-bit	+0.012 [−0.018, +0.042]	+0.000 [−0.030, +0.030]	−0.009 [−0.055, +0.036]	
Llama-8B, 4-bit vs 16-bit	−0.027 [−0.061, +0.008]	−0.025 [−0.057, +0.007]	−0.018 [−0.065, +0.028]	
DeepSeek-7B, 4-bit vs 16-bit	−0.013 [−0.052, +0.025]	−0.030 [−0.065, +0.005]	+0.017 [−0.032, +0.067]	
Qwen-7B, 4-bit vs 16-bit	−0.028 [−0.067, +0.012]	−0.052 [−0.090, −0.015]*	−0.053 [−0.112, +0.004]	
Base-model quantisation, combined configurations (3 vs 3 seeds)				
Llama-8B, 4-bit vs 16-bit	+0.016 [−0.003, +0.035]	+0.005 [−0.010, +0.019]	+0.007 [−0.018, +0.034]	
DeepSeek-7B, 4-bit vs 16-bit	+0.010 [−0.010, +0.031]	−0.003 [−0.021, +0.014]	+0.019 [−0.010, +0.049]	
Qwen-7B, 4-bit vs 16-bit	+0.016 [−0.004, +0.035]	+0.003 [−0.012, +0.019]	+0.023 [−0.005, +0.052]	
Qwen-1.5B, 4-bit vs 16-bit	+0.002 [−0.020, +0.025]	+0.003 [−0.015, +0.022]	+0.015 [−0.017, +0.048]	
Higher rank at 10^{-4}: combined vs r=8 (3 vs 3 seeds)				
Llama-8B, r=32	−0.006 [−0.028, +0.015]	+0.001 [−0.016, +0.017]	−0.007 [−0.038, +0.022]	
DeepSeek-7B, r=64	+0.013 [−0.012, +0.037]	+0.003 [−0.018, +0.025]	+0.013 [−0.018, +0.044]	
Qwen-7B, r=32	+0.013 [−0.011, +0.037]	+0.000 [−0.018, +0.018]	+0.026 [−0.008, +0.061]	
Pruning with re-masked recovery vs dense (Llama, seed 42)				
Wanda 10%	−0.020 [−0.053, +0.012]	−0.020 [−0.047, +0.005]	+0.003 [−0.046, +0.051]	
Wanda 30%	−0.026 [−0.062, +0.008]	+0.003 [−0.027, +0.032]	+0.008 [−0.046, +0.061]	
Wanda 50%	−0.101 [−0.132, −0.071]*	−0.032 [−0.057, −0.004]*	−0.136 [−0.184, −0.090]*	yes
Wanda 2:4	−0.266 [−0.301, −0.231]*	−0.214 [−0.250, −0.179]*	−0.410 [−0.462, −0.360]*	yes
Wanda 4:8	−0.115 [−0.146, −0.085]*	−0.056 [−0.083, −0.028]*	−0.163 [−0.209, −0.118]*	yes
magnitude 10%	−0.030 [−0.063, +0.002]	−0.002 [−0.032, +0.027]	−0.018 [−0.069, +0.031]	
magnitude 30%	−0.062 [−0.100, −0.024]*	−0.010 [−0.042, +0.022]	−0.018 [−0.075, +0.040]	yes
magnitude 50%	−0.289 [−0.333, −0.245]*	−0.402 [−0.460, −0.347]*	−0.573 [−0.628, −0.521]*	yes
magnitude 2:4	−0.218 [−0.262, −0.174]*	−0.112 [−0.155, −0.070]*	−0.319 [−0.383, −0.252]*	yes
magnitude 4:8	−0.191 [−0.232, −0.150]*	−0.150 [−0.193, −0.108]*	−0.290 [−0.349, −0.233]*	yes
Mask-aware vs re-masked recovery (r=8, 5×10^{-5})				
Wanda 50%	+0.072 [+0.041, +0.104]*	+0.009 [−0.022, +0.039]	+0.063 [+0.014, +0.114]*	yes
Wanda 2:4	+0.180 [+0.141, +0.218]*	+0.172 [+0.134, +0.210]*	+0.271 [+0.222, +0.319]*	yes
Wanda 4:8	+0.046 [+0.014, +0.077]*	+0.006 [−0.025, +0.036]	+0.058 [+0.011, +0.105]*	yes
magnitude 50%	+0.233 [+0.186, +0.280]*	+0.372 [+0.312, +0.432]*	+0.459 [+0.405, +0.512]*	yes
magnitude 2:4	+0.149 [+0.104, +0.193]*	+0.070 [+0.025, +0.117]*	+0.197 [+0.129, +0.263]*	yes
magnitude 4:8	+0.149 [+0.108, +0.190]*	+0.115 [+0.070, +0.160]*	+0.218 [+0.161, +0.276]*	yes
Mask-aware recovery (r=8, 5×10^{-5}) vs dense				
Wanda 50%	−0.029 [−0.063, +0.005]	−0.022 [−0.055, +0.010]	−0.073 [−0.127, −0.019]*	
Wanda 2:4	−0.086 [−0.126, −0.047]*	−0.042 [−0.077, −0.008]*	−0.140 [−0.191, −0.088]*	yes
Wanda 4:8	−0.069 [−0.102, −0.037]*	−0.050 [−0.080, −0.020]*	−0.105 [−0.151, −0.058]*	yes
magnitude 50%	−0.055 [−0.092, −0.019]*	−0.030 [−0.068, +0.005]	−0.114 [−0.164, −0.064]*	yes
magnitude 2:4	−0.069 [−0.108, −0.030]*	−0.042 [−0.078, −0.008]*	−0.122 [−0.179, −0.064]*	yes
magnitude 4:8	−0.042 [−0.078, −0.006]*	−0.035 [−0.068, −0.003]*	−0.072 [−0.120, −0.024]*	yes
Mask-aware recovery at 10^{-4} (r=8: single runs; r=32: 3 vs 3 seeds)				
Wanda 50%, r=8 vs dense 5×10^{-5}	−0.018 [−0.052, +0.016]	−0.005 [−0.035, +0.025]	−0.029 [−0.082, +0.023]	
Wanda 50%, r=8 vs dense 10^{-4}	−0.064 [−0.095, −0.034]*	−0.032 [−0.057, −0.007]*	−0.095 [−0.144, −0.050]*	yes
Wanda 50%, r=8 vs mask-aware 5×10^{-5}	+0.011 [−0.018, +0.041]	+0.017 [−0.012, +0.047]	+0.044 [−0.007, +0.095]	
Wanda 2:4, r=8 vs dense 5×10^{-5}	−0.024 [−0.059, +0.011]	−0.017 [−0.050, +0.015]	−0.043 [−0.092, +0.006]	
Wanda 2:4, r=8 vs dense 10^{-4}	−0.070 [−0.105, −0.036]*	−0.045 [−0.075, −0.015]*	−0.109 [−0.159, −0.061]*	yes
Wanda 2:4, r=8 vs mask-aware 5×10^{-5}	+0.062 [+0.024, +0.100]*	+0.025 [−0.010, +0.060]	+0.097 [+0.047, +0.147]*	yes
Wanda 4:8, r=8 vs dense 5×10^{-5}	+0.010 [−0.021, +0.042]	+0.010 [−0.015, +0.035]	−0.008 [−0.061, +0.045]	
Wanda 4:8, r=8 vs dense 10^{-4}	−0.036 [−0.067, −0.006]*	−0.017 [−0.040, +0.005]	−0.075 [−0.125, −0.027]*	no
Wanda 4:8, r=8 vs mask-aware 5×10^{-5}	+0.079 [+0.045, +0.114]*	+0.060 [+0.030, +0.092]*	+0.096 [+0.046, +0.148]*	yes
Wanda 50%, r=32 vs dense 5×10^{-5}	+0.029 [+0.008, +0.051]*	+0.017 [+0.000, +0.033]	+0.043 [+0.011, +0.077]*	no
Wanda 50%, r=32 vs dense 10^{-4}	−0.007 [−0.026, +0.012]	+0.004 [−0.012, +0.021]	−0.028 [−0.056, +0.001]	
Wanda 50%, r=32 vs mask-aware 5×10^{-5}	+0.064 [+0.034, +0.094]*	+0.037 [+0.011, +0.063]*	+0.115 [+0.070, +0.159]*	yes
Wanda 2:4, r=32 vs dense 5×10^{-5}	−0.021 [−0.046, +0.004]	−0.014 [−0.034, +0.006]	−0.026 [−0.063, +0.010]	
Wanda 2:4, r=32 vs dense 10^{-4}	−0.058 [−0.081, −0.034]*	−0.027 [−0.047, −0.007]*	−0.097 [−0.134, −0.061]*	yes
Wanda 2:4, r=32 vs mask-aware 5×10^{-5}	+0.070 [+0.037, +0.103]*	+0.026 [−0.004, +0.057]	+0.112 [+0.069, +0.154]*	yes
Wanda 4:8, r=32 vs dense 5×10^{-5}	+0.016 [−0.008, +0.039]	+0.014 [−0.006, +0.034]	+0.020 [−0.014, +0.056]	
Wanda 4:8, r=32 vs dense 10^{-4}	−0.021 [−0.043, +0.001]	+0.002 [−0.018, +0.022]	−0.051 [−0.084, −0.018]*	
Wanda 4:8, r=32 vs mask-aware 5×10^{-5}	+0.090 [+0.059, +0.121]*	+0.062 [+0.034, +0.091]*	+0.123 [+0.080, +0.166]*	yes
Wanda 50%, r=32 vs combined	−0.001 [−0.021, +0.020]	+0.003 [−0.012, +0.018]	−0.021 [−0.051, +0.010]	
Wanda 2:4, r=32 vs combined	−0.052 [−0.078, −0.025]*	−0.028 [−0.048, −0.008]*	−0.090 [−0.126, −0.054]*	yes
Wanda 4:8, r=32 vs combined	−0.015 [−0.039, +0.009]	+0.001 [−0.018, +0.020]	−0.044 [−0.076, −0.010]*	
1–1.5B model (r=32, 10^{-4}) vs 7–8B model of the same family (r=8; 3 vs 3 seeds)				
Llama-1B vs Llama-8B at 10^{-4}	−0.104 [−0.130, −0.078]*	−0.052 [−0.075, −0.030]*	−0.196 [−0.235, −0.158]*	yes
Llama-1B vs Llama-8B at 5×10^{-5}	−0.067 [−0.093, −0.041]*	−0.040 [−0.062, −0.018]*	−0.125 [−0.164, −0.087]*	yes

Comparison	Δ F1-score [95% CI]	Δ Acc-P [95% CI]	Δ Acc-Args [95% CI]	Holm
DeepSeek-1.5B vs DeepSeek-7B at 10^{-4}	-0.082 [-0.112, -0.051]*	-0.027 [-0.054, -0.001]*	-0.117 [-0.155, -0.079]*	yes
DeepSeek-1.5B vs DeepSeek-7B at 5×10^{-5}	$+0.007$ [-0.023, $+0.038$]	$+0.006$ [-0.023, $+0.034$]	$+0.020$ [-0.016, $+0.057$]	
Qwen-1.5B vs Qwen-7B at 10^{-4}	-0.034 [-0.057, -0.011]*	-0.022 [-0.041, -0.002]*	-0.071 [-0.107, -0.035]*	yes
Qwen-1.5B vs Qwen-7B at 5×10^{-5}	-0.000 [-0.025, $+0.024$]	-0.008 [-0.028, $+0.012$]	-0.018 [-0.054, $+0.017$]	
Distillation vs direct fine-tuning of the 1–1.5B model (3 vs 3 seeds)				
Llama-1B, distilled vs direct	$+0.017$ [-0.004, $+0.040$]	$+0.026$ [$+0.005$, $+0.047$]*	$+0.036$ [$+0.004$, $+0.068$]*	
Llama-1B, filtered vs direct	$+0.021$ [$+0.001$, $+0.041$]*	$+0.025$ [$+0.006$, $+0.045$]*	$+0.035$ [$+0.006$, $+0.065$]*	no
Llama-1B, filtered vs unfiltered	$+0.003$ [-0.019, $+0.026$]	-0.001 [-0.019, $+0.017$]	-0.000 [-0.033, $+0.032$]	
DeepSeek-1.5B, distilled vs direct	-0.002 [-0.028, $+0.025$]	-0.005 [-0.028, $+0.019$]	-0.008 [-0.041, $+0.025$]	
DeepSeek-1.5B, filtered vs direct	$+0.011$ [-0.015, $+0.037$]	$+0.006$ [-0.018, $+0.029$]	$+0.006$ [-0.026, $+0.037$]	
DeepSeek-1.5B, filtered vs unfiltered	$+0.013$ [-0.013, $+0.039$]	$+0.011$ [-0.013, $+0.035$]	$+0.014$ [-0.016, $+0.043$]	
Qwen-1.5B, distilled vs direct	-0.013 [-0.037, $+0.010$]	-0.001 [-0.020, $+0.018$]	-0.014 [-0.051, $+0.023$]	
Qwen-1.5B, filtered vs direct	$+0.003$ [-0.019, $+0.025$]	$+0.001$ [-0.017, $+0.018$]	-0.001 [-0.034, $+0.034$]	
Qwen-1.5B, filtered vs unfiltered	$+0.016$ [-0.005, $+0.038$]	$+0.002$ [-0.017, $+0.021$]	$+0.014$ [-0.020, $+0.047$]	
Decision-first output order (Llama, r=8, 10^{-4}; 3 vs 3 seeds)				
decision first vs thought first	-0.018 [-0.039, $+0.003$]	-0.002 [-0.022, $+0.017$]	-0.038 [-0.067, -0.009]*	

The learning-rate effect is significant for all three models (+0.037, +0.089 and +0.033, averaged over three seeds). Among the pruning settings with re-masked recovery, Wanda at 50% sparsity, all semi-structured settings and magnitude pruning at 30% and 50% lie clearly below the dense model; mask-aware recovery improves all six settings at 50% sparsity significantly, and Wanda at 50% is then no longer significantly below the dense model. With the paper's learning rate, none of the six mask-aware configurations (Wanda at 50%, 2:4 and 4:8, each with $r = 8$ and $r = 32$; three seeds for $r = 32$) is significantly below the dense model at 5×10^{-5}, and Wanda at 50% with $r = 32$ is nominally above it ($+0.029$, $[0.008, 0.051]$); against the dense model at 10^{-4}, the $r = 32$ configurations at 50% and 4:8 show no significant difference in F1-score, whereas all three $r = 8$ runs and 2:4 with $r = 32$ remain below. All quantisation comparisons, at the released default and at the combined configuration (three seeds each) and for Qwen2.5-1.5B (4-bit versus three 16-bit seeds), the higher rank of the combined configurations compared with $r = 8$ at 10^{-4} (three seeds each), as well as Wanda at 10% and 30% and magnitude pruning at 10%, have confidence intervals that include zero: these differences are not significant. Equivalence within ± 0.03 is shown for 8-bit Llama, for the 4-bit combined configuration of DeepSeek and for the exact released configuration of DeepSeek and Qwen; the other intervals extend beyond the margin, so we report these results as no significant difference rather than as equivalence.

With the Holm–Bonferroni correction within each claim family (Section 5.6), every F1-score effect on which our conclusions rest remains significant: the learning-rate effect for all three models, the losses of all re-masked pruning settings except Wanda and magnitude at 10% and Wanda at 30%, the gains of mask-aware over re-masked recovery for all six settings, the remaining losses of the mask-aware settings at 5×10^{-5} (except Wanda at 50%), the losses of mask-aware recovery at 10^{-4} against the dense model at 10^{-4} for Wanda at 50% and 2:4 with $r = 8$ and for 2:4 with $r = 32$, the lower Acc-Args of 4:8 with $r = 32$ against the dense model at 10^{-4} (-0.051), and the gain of the validation-selected Llama configuration over the combined configuration ($+0.025$). The following nominally significant effects do not survive the correction and are therefore not claimed: the lower Acc-P of 4-bit Qwen2.5-7B (-0.052); the F1-score gain of mask-aware Wanda at 50% ($r = 32$, 10^{-4}) over the dense model at 5×10^{-5} ($+0.029$); the F1-score loss of mask-aware 4:8 with $r = 8$ at 10^{-4} against the dense model at 10^{-4} (-0.036); all gains of distillation for Llama-3.2-1B (Table 6); the gain of the validation-selected Qwen configuration over its combined configuration ($+0.021$); and several secondary Acc-P and Acc-Args differences of pruned models (Acc-P: re-masked Wanda at 50%; mask-aware Wanda 2:4, magnitude 2:4 and magnitude 4:8 at 5×10^{-5}; mask-aware 2:4 with $r = 32$ and Wanda at 50% with $r = 8$ at 10^{-4} against the dense model).

To test whether the ranking of methods reproduces, we compare our F1-scores in Table 1 with those reported by Yang et al. (2025b) for the five methods and three models. Kendall's τ is 0.80 for each model ($p = 0.08$, with only five methods per model) and 0.82 over all 15 method–model pairs ($p < 0.001$).

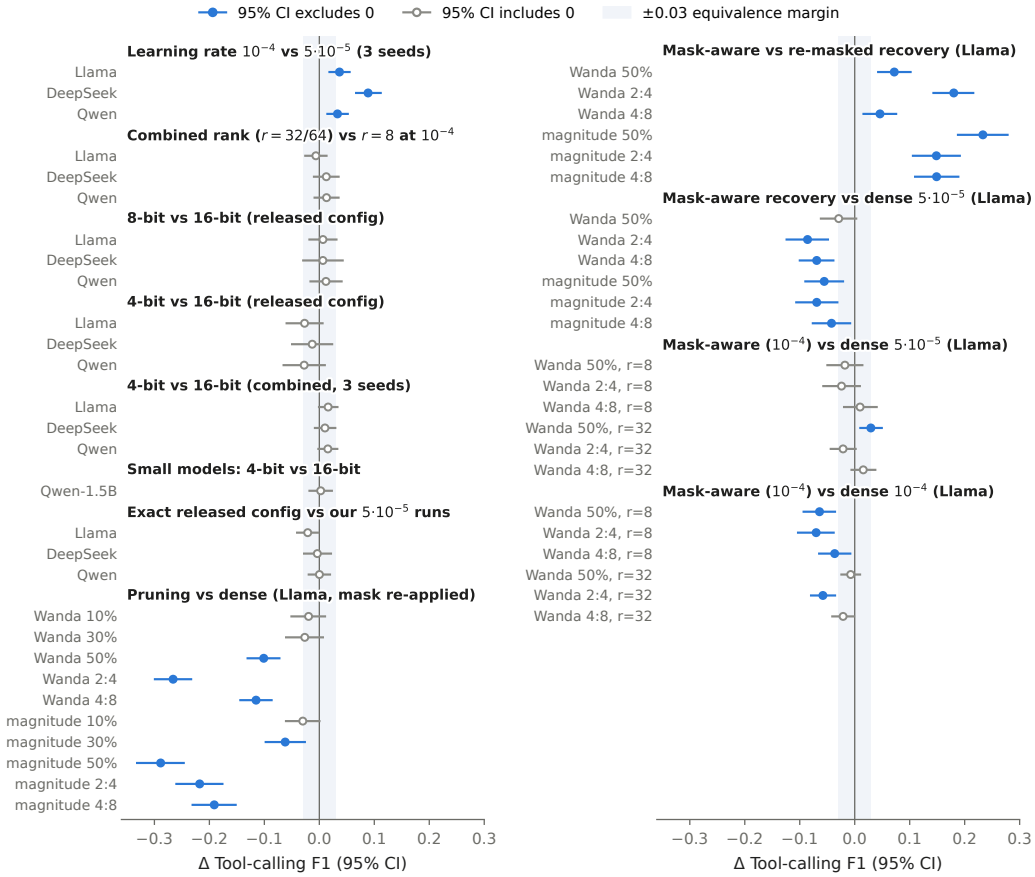


Figure 3: Paired-bootstrap differences in tool-calling F1-score with 95% confidence intervals over the 400 test samples. Filled blue: the interval excludes zero; hollow grey: it includes zero (no significant difference). Shaded band: ±0.03. Learning-rate, combined-configuration and rank comparisons average three seeds per configuration; pruning comparisons are against the seed-42 dense model; mask-aware recovery comparisons at 5×10^{-5} are single runs. The last groups compare mask-aware recovery at learning rate 10^{-4} with the dense model at 5×10^{-5} and at 10^{-4} ($r = 8$: single runs against the seed-42 dense model; $r = 32$: three seeds against three dense seeds), and 4-bit with 16-bit Qwen2.5-1.5B (three seeds each).

F Where the Models Fail

F.1 Error cascade

Table 27 gives the error cascade of Section 6.3 for all configurations. Every test sample is assigned to exactly one stage of the decision pipeline: (a) no parseable proactive score; (b) a parseable but wrong proactive decision (missed or false detection); (c) a correct decision that no assistance is needed; (d) a correct proactive trigger with a wrong or incomplete tool set; (e) a correct trigger and the exact tool set, but at least one wrong tool argument (the rule of Acc-Args: the tool is not in the ground-truth chain or its sandbox call returns an error); and (f) a fully correct response. With several runs per configuration, the shares are averaged over runs.

Table 27: Error cascade on the 400 test samples (% of samples; each sample is counted in exactly one stage). (a) no proactive score; (b) wrong proactive decision; (c) correct non-proactive decision; (d) correct trigger, wrong tool set; (e) correct trigger and tool set, wrong arguments; (f) fully correct. LoRA rows: $r = 8$, mean over three seeds; pruning rows: `Llama-3.1-8B-Instruct` with the pruning mask re-applied, mean over the repeated evaluations where available; ‡mask-aware recovery, rows marked 10^{-4} with learning rate 10^{-4} and the stated rank; mask-aware rows with $r = 32$ and all 1–1.5B rows: mean over three seeds.

Model	Configuration	Runs	(a)	(b)	(c)	(d)	(e)	(f)
Llama-3.1-8B-Ins	ICL-All	1	12.8	20.2	35.2	28.2	0.0	3.5
	LoRA 5×10^{-5} (code)	3	0.0	8.5	42.9	34.2	0.0	14.4
	LoRA 10^{-4} (paper)	3	0.0	7.2	43.8	32.0	0.0	17.0
	4-bit, 5×10^{-5}	1	0.0	10.8	40.0	36.8	0.0	12.5
DeepSeek-R1-7B	ICL-All	1	55.5	8.8	6.5	27.8	0.5	1.0
	LoRA 5×10^{-5} (code)	3	0.0	14.5	38.1	39.9	0.0	7.5
	LoRA 10^{-4} (paper)	3	0.1	11.1	40.7	35.1	0.1	13.0
	4-bit, 5×10^{-5}	1	0.0	15.8	37.2	39.2	0.0	7.8
Qwen2.5-7B-Ins	ICL-All	1	40.8	6.5	9.2	38.5	0.2	4.8
	LoRA 5×10^{-5} (code)	3	0.1	8.8	42.2	37.4	0.0	11.5
	LoRA 10^{-4} (paper)	3	0.0	7.5	43.9	34.2	0.0	14.3
	4-bit, 5×10^{-5}	1	20.0	8.2	27.8	32.8	0.0	11.2
Llama-3.1-8B-Ins (pruned)	Wanda 10% / 30% / 50%	1/1/3	0.0 / 0.0 / 0.2	10.2 / 8.0 / 11.2	40.0 / 43.8 / 42.2	34.0 / 37.5 / 38.2	0.0 / 0.0 / 0.0	15.8 / 10.8 / 8.1
	Magnitude 10% / 30% / 50%	1	0.0 / 0.0 / 92.5	8.5 / 9.2 / 1.5	42.8 / 40.8 / 1.2	35.8 / 38.2 / 4.8	0.0 / 0.0 / 0.0	13.0 / 11.8 / 0.0
	Wanda 2:4 / 4:8	3	34.0 / 0.9	15.7 / 13.2	13.5 / 42.2	33.1 / 36.0	0.9 / 0.3	2.8 / 7.2
	Magnitude 2:4 / 4:8	1	3.0 / 6.2	17.0 / 18.8	35.0 / 38.2	41.5 / 34.0	1.0 / 0.2	2.5 / 2.5
	Wanda 50% / 2:4 / 4:8‡	1	0.0 / 0.0 / 0.5	10.5 / 12.5 / 12.8	43.8 / 39.0 / 42.0	34.5 / 39.8 / 34.8	0.0 / 0.2 / 0.0	11.2 / 8.5 / 10.0
	Wanda 50% / 2:4 / 4:8‡, 10^{-4}, r=8	1	0.2 / 0.2 / 0.8	8.8 / 9.8 / 7.2	43.8 / 41.2 / 43.2	34.5 / 36.0 / 33.5	0.0 / 0.0 / 0.0	12.8 / 12.8 / 15.2
	Wanda 50% / 2:4 / 4:8‡, 10^{-4}, r=32	3	0.1 / 1.7 / 0.0	6.8 / 9.4 / 7.1	43.5 / 41.2 / 44.0	33.3 / 33.3 / 34.2	0.0 / 0.0 / 0.0	16.2 / 14.3 / 14.7
	Magnitude 50% / 2:4 / 4:8‡	1	0.5 / 0.2 / 0.0	11.0 / 12.2 / 11.8	40.5 / 41.8 / 41.5	37.0 / 35.0 / 37.0	0.0 / 0.0 / 0.0	11.0 / 10.8 / 9.8
	Structured activation 10% / 30% / 50%	1	0.0 / 0.5 / 0.5	13.0 / 28.2 / 47.5	42.2 / 35.5 / 30.5	38.0 / 33.5 / 21.5	0.0 / 0.0 / 0.0	6.8 / 2.2 / 0.0
	Structured magnitude 10% / 30% / 50%	1	100 / 100 / 100	0	0	0	0	0
1–1.5B, 2048 tokens	Llama-3.2-1B, $r = 32$	3	0.1	12.5	41.8	35.2	0.0	10.3
	DeepSeek-R1-1.5B, $r = 32$	3	0.3	13.8	39.5	39.2	0.2	6.9
	Qwen2.5-1.5B, $r = 32$	3	0.0	9.7	43.1	35.9	0.0	11.3
	Qwen2.5-1.5B, $r = 32$, 4-bit	3	0.1	9.3	42.8	35.5	0.0	12.3

Three findings stand out. First, once a model is fine-tuned, practically all of its errors are tool-selection errors: stage (e) stays at or below 0.1% for every fine-tuned 16-bit 7–8B model (and at or below 0.2% for the 1–1.5B models), so a correct tool set almost always comes with correct arguments, and the differences in Acc-Args between configurations stem from choosing the wrong tools, not from filling them in wrongly. The paper's learning rate improves all three models in the same way, moving samples from (b) and (d) to (f) (fully correct: Llama 14.4% to 17.0%, DeepSeek 7.5% to 13.0%, Qwen 11.5% to 14.3%). Second, the in-context baselines fail mainly on the output format: 55.5% (DeepSeek) and 40.8% (Qwen) of their outputs contain no parseable proactive score, which fine-tuning removes completely. Third, the compressed models fail in different ways. With the original, re-masked recovery, 50% magnitude pruning and Wanda 2:4 cause a *format collapse* (no parseable score for 92.5% and 34% of the samples); mask-aware recovery removes it (0.5% and 0%), so the collapse is caused by the recovery procedure rather than by the sparsity. With the paper's learning rate and $r = 32$, mask-aware recovery moves further samples from (b) to (f) (fully correct: 16.2% for Wanda at 50%, close to the 17.0% of the dense model at 10^{-4}); the only exception is 2:4, where on average 1.7% of the outputs again lack a parseable score (4.0% for seed 42). Structured magnitude pruning loses the format completely (100%) at every ratio, structured activation pruning mainly causes *wrong proactive decisions* (stage (b) rising from 13.0% to 28.2% and 47.5%), and moderate unstructured pruning, mask-aware recovery and the 1–1.5B models keep the decisions intact and lose mainly on *tool selection*. Quantisation mostly behaves like the 16-bit model, with one exception: 4-bit Qwen at the released learning rate produces no parseable score on 20% of the samples, which explains its lower Acc-P (nominally significant, but not after the Holm correction; Section 6.2.2); the 4-bit Qwen2.5-1.5B with $r = 32$ shows no such failure (0.1%, three seeds).

Table 28: Tool-calling F1-score and Acc-Args by the length of the ground-truth tool chain. For samples that need no tool (187 of 400), F1 is the share of correct "no tool" predictions and Acc-Args is undefined. LoRA rows: $r = 8$, mean over three seeds; ‡mask-aware recovery (10^{-4}: learning rate 10^{-4}, $r = 32$, mean over three seeds); Qwen2.5-1.5B: mean over three seeds.

		0 tools ($n = 187$)	1 tool ($n = 72$)		2 tools ($n = 65$)		≥3 tools ($n = 76$)	
Model	Configuration	F1	F1	Acc-Args	F1	Acc-Args	F1	Acc-Args
Llama-3.1-8B-Ins	ICL-All	0.930	0.227	0.231	0.278	0.357	0.197	0.341
	LoRA 5×10^{-5}	0.913	0.588	0.454	0.587	0.643	0.591	0.738
	LoRA 10^{-4}	0.930	0.651	0.530	0.623	0.691	0.650	0.806
DeepSeek-R1-7B	ICL-All	0.909	0.097	0.088	0.183	0.209	0.117	0.273
	LoRA 5×10^{-5}	0.818	0.374	0.274	0.422	0.428	0.409	0.523
	LoRA 10^{-4}	0.870	0.485	0.355	0.546	0.563	0.537	0.698
Qwen2.5-7B-Ins	ICL-All	0.914	0.259	0.273	0.387	0.484	0.244	0.417
	LoRA 5×10^{-5}	0.893	0.492	0.365	0.558	0.597	0.581	0.740
	LoRA 10^{-4}	0.930	0.527	0.401	0.584	0.641	0.607	0.767
Llama (pruned)	Wanda 30%	0.925	0.498	0.443	0.524	0.589	0.533	0.780
	Wanda 50%	0.888	0.466	0.380	0.357	0.420	0.405	0.681
	Wanda 50%‡	0.930	0.493	0.395	0.537	0.566	0.498	0.686
	Wanda 50%‡, 10^{-4}	0.925	0.604	0.457	0.644	0.670	0.652	0.811
	Structured activation 10%	0.893	0.368	0.272	0.436	0.450	0.434	0.577
	Structured activation 30%	0.765	0.150	0.114	0.248	0.285	0.298	0.438
Qwen2.5-1.5B-Ins	LoRA $r = 32$, 10^{-4}	0.916	0.480	0.358	0.541	0.554	0.548	0.723

Table 28 separates the samples by the number of tools the ground truth requires. Almost all configurations recognise most samples that need no tool (F1 0.82–0.94; only structured activation pruning at 30% drops to 0.77), so the differences between configurations come mainly from the 213 samples that require tools. There, the in-context baselines reach an F1-score of only 0.10–0.39 regardless of chain length, whereas the fine-tuned models score 0.37–0.65 with little dependence on the length of the chain. The gain from the paper's learning rate is spread over all chain lengths: for Llama it is largest for single-tool and long chains (+0.06 each), and for DeepSeek, which gains most, it amounts to +0.11 to +0.13 at every length. Acc-Args rises with the chain length for every fine-tuned model (e.g. Llama at 10^{-4}: 0.53, 0.69, 0.81), because the metric is computed per tool call: a missed proactive action counts as a single wrong call, which weighs more in a sample with one ground-truth tool than in a sample with three or more. With re-masked recovery, pruning hurts the longer chains first: at 50% Wanda sparsity, F1 for two-tool chains falls from 0.524 (30%) to 0.357, while single-tool chains fall only from 0.498 to 0.466; mask-aware recovery restores the two-tool chains to 0.537, and with the paper's learning rate and $r = 32$ it matches the dense model at 10^{-4} for two and more tools (0.644 versus 0.623 and 0.652 versus 0.650), but stays below it for single-tool chains (0.604 versus 0.651). Qwen2.5-1.5B with $r = 32$ stays below Qwen2.5-7B at 10^{-4} at every chain length, by 0.04–0.06 in F1-score for the samples that require tools, most for long chains (0.548 versus 0.607).

F.2 Argument Values

The released Acc-Args metric never compares argument values with the reference (Section 5.6). Table 29 checks them directly. For each of the 289 ground-truth tool calls with arguments in the test split, we look for a predicted call of the same tool in the same sample and compare its arguments: the same argument keys, all values equal after normalisation (lower case, whitespace and surrounding punctuation removed), and the token-level F1-score of the values, averaged over the reference keys. Once the right tool is chosen, the keys are almost always right (≥ 0.99 for every fine-tuned model), while the values match the reference exactly in only about a fifth of the calls, with a token-level F1-score of 0.50–0.59 for the fine-tuned 7–8B models. String matching, however, strongly understates argument quality, because many arguments are free text, such as search queries or event titles, for which the reference is only one valid wording. In a blind sample of 150 of these calls, stratified over all configurations and labelled by one of the authors (correct, partial or incorrect, given the situation, the persona and the reference), 84% of the predicted arguments were judged correct and 94% at least partially correct, although only 16% match the reference exactly and their mean

token-level F1-score is 0.51; the token-level F1-score separates correct from other arguments only moderately (AUC 0.74). Once a model has chosen the right tool, it therefore almost always fills it in sensibly, and the differences in Acc-Args between configurations reflect tool selection rather than argument quality. Consistent with this, the learning rate mainly changes how often the right tool is found (e.g. Llama 0.53 to 0.59), not the token-level similarity of its arguments (0.56 versus 0.58), and the same holds for the decision-first order. In-context learning finds the right tool far less often (0.11–0.30).

Argument check: add the open-judge results per configuration (Qwen2.5-14B-Instruct-AWQ, `scripts/argument_judge.py summary`), judge–human agreement (Cohen's κ) on the 150 labelled calls, and a second annotator if available.

Table 29: Argument values of the predicted tool calls (289 ground-truth calls with arguments). Tool found: share of these calls whose tool is predicted in the same sample; the other columns refer to the calls whose tool is found. Mean ± standard deviation over runs (seeds or repeated evaluations). ‡Mask-aware recovery. Script: `scripts/argument_values.py`.

Model	Configuration	Runs	Tool found	Same keys	Exact values	Value token-F1
Llama-3.1-8B-Ins	ICL-All	1	0.176	1.000	0.020	0.299
	LoRA 5×10^{-5}	3	0.532±0.016	0.993±0.000	0.211±0.009	0.564±0.004
	LoRA 10^{-4}	3	0.593±0.011	0.994±0.000	0.231±0.016	0.582±0.003
	Decision first, 10^{-4}	3	0.593±0.019	0.994±0.000	0.241±0.021	0.592±0.015
	Wanda 50%, re-masked	3	0.311±0.021	0.993±0.006	0.126±0.020	0.427±0.021
	Wanda 50%‡, 10^{-4}, r=32	3	0.615±0.011	0.991±0.007	0.205±0.025	0.543±0.017
DeepSeek-R1-7B	ICL-All	1	0.114	0.636	0.030	0.242
	LoRA 5×10^{-5}	3	0.356±0.018	0.990±0.000	0.181±0.007	0.502±0.026
	LoRA 10^{-4}	3	0.484±0.027	0.993±0.000	0.199±0.022	0.534±0.010
Qwen2.5-7B-Ins	ICL-All	1	0.298	1.000	0.070	0.423
	LoRA 5×10^{-5}	3	0.506±0.031	0.996±0.004	0.196±0.016	0.555±0.019
	LoRA 10^{-4}	3	0.529±0.016	0.993±0.000	0.192±0.010	0.570±0.017
1–1.5B, $r = 32$	Llama-3.2-1B	3	0.451±0.012	0.987±0.009	0.170±0.053	0.501±0.037
	DeepSeek-R1-1.5B	3	0.358±0.012	0.987±0.011	0.145±0.037	0.458±0.050
	Qwen2.5-1.5B	3	0.498±0.043	0.996±0.004	0.212±0.009	0.562±0.020

G Robustness Across Scenarios

All results in the main text aggregate the 400 test samples over the nine scenarios of *ContextAgentBench*. To test whether the efficiency techniques degrade performance uniformly or whether some scenarios collapse, Figure 4 breaks the tool-calling F1-score down by scenario (31–54 test samples each). The scenarios differ clearly in difficulty: for the dense Llama model, Health (0.58) and Chitchat (0.62) are the hardest and Leisure (0.85) and Work (0.86) the easiest, and this ordering is largely shared by all configurations. For each technique we compute the per-scenario F1 difference to its reference with a paired bootstrap and test for heterogeneity with a permutation test on the between-scenario variance of the per-sample differences (10,000 permutations of the scenario labels).

4-bit quantisation of the combined configuration changes the F1-score by between −0.027 (Travel) and +0.039 (Leisure) per scenario, with no evidence of heterogeneity ($p = 0.79$). Wanda pruning to 50% sparsity with re-masked recovery lowers the F1-score in every scenario; the loss ranges from −0.034 (Leisure) to −0.173 (Cooking) and is larger in Chitchat, Cooking, Shopping and Travel than in Leisure and Outdoors, but these differences are compatible with a uniform effect ($p = 0.34$). With mask-aware recovery, the overall loss shrinks to −0.029, again without heterogeneity ($p = 0.44$). Replacing Qwen2.5-7B by Qwen2.5-1.5B at the same configuration costs −0.047 overall, also without significant heterogeneity ($p = 0.15$). With 31–54 samples per scenario, the per-scenario confidence intervals are wide (about ±0.04 to ±0.14), so moderate scenario-specific effects cannot be excluded, but none of the techniques causes a scenario to collapse while others remain intact.

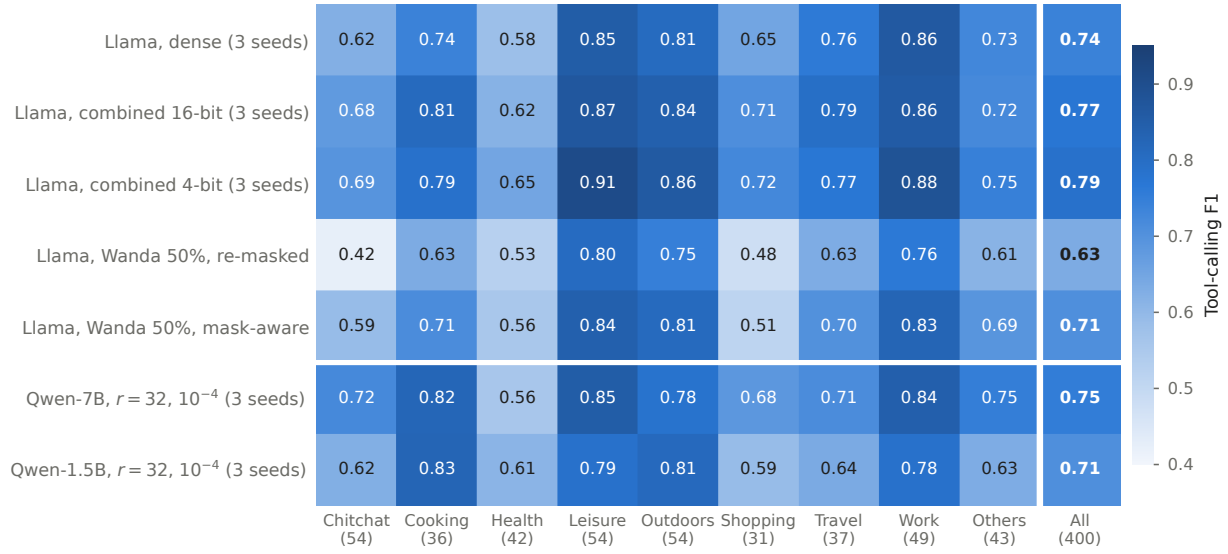


Figure 4: Tool-calling F1-score per *ContextAgentBench* scenario (number of test samples in brackets). Rows with three seeds or repeated evaluations show the mean. The last column repeats the overall score.